第三章 组合逻辑电路

组合逻辑电路（Combinational Logic Circuit）是数字系统中最基础、应用最广泛的电路类型之一。其核心特征在于：电路在任意时刻的输出状态，完全由该时刻各输入端的逻辑状态唯一确定，与电路的历史状态无关。这意味着组合逻辑电路不具有记忆功能，电路内部不包含任何存储单元，也不存在从输出端到输入端的反馈路径。

正是由于这一基本特征，组合逻辑电路的行为可以完整地用布尔代数来描述和分析。对于拥有 n 个输入变量的组合逻辑电路，其真值表包含 2^n行，每一行对应一种唯一的输入取值组合，以及对应的确定输出值。这种确定性和可枚举性，使得组合逻辑电路的分析与设计具有严格的规律可循。

在现代数字系统中，组合逻辑电路无处不在。微处理器中的算术逻辑单元（ALU）执行加减法和逻辑运算的核心通路是组合逻辑电路；存储器芯片内部的地址译码器是组合逻辑电路；总线系统中的仲裁逻辑、优先级判断电路是组合逻辑电路；FPGA 芯片中的查找表（LUT，Look-Up Table）本质上就是一种可编程的组合逻辑器件。可以说，理解和掌握组合逻辑电路是数字系统设计的根本基础。

本章系统介绍组合逻辑电路的分析方法（3.2 节）、设计方法（3.3 节），以及编码器、译码器、数据选择器、加法器、数值比较器等若干常用组合逻辑器件（3.4 节），最后讨论组合逻辑电路中竞争-冒险现象的产生机理与消除方法（3.5 节）。通过本章的学习，应能系统化地分析与设计任意组合逻辑电路，并能熟练运用各类中规模集成（MSI）器件完成数字系统的设计任务。

3.1 概述

在正式学习组合逻辑电路的分析与设计方法之前，有必要对组合逻辑电路的基本概念、结构特征和分类方式作一个系统性概述，为后续学习奠定清晰的认知基础。

3.1.1 组合逻辑电路的基本特征

组合逻辑电路具有以下三个本质特征，这三个特征相互关联、缺一不可，共同定义了组合逻辑电路区别于时序逻辑电路的根本属性：

1. 无记忆性：组合逻辑电路不包含任何存储单元（触发器、锁存器等），电路没有内部状态，输出完全由当前输入决定，与过去的输入历史无关。一旦输入发生变化，输出（经过有限的传播延迟后）也会随之改变到新的稳定值。

2. 无反馈路径：组合逻辑电路的信号传播方向是单向的，从输入端流向输出端，不存在从输出端到输入端的信号反馈路径。若电路中存在反馈，则会引入时序特性，使电路行为依赖于历史状态，从而不再是组合逻辑电路。
3. 确定性输出：对于每一种确定的输入变量取值组合，组合逻辑电路必然产生唯一确定的输出值。这种一一对应的关系可以用完整的真值表来描述，也可以用布尔函数来表示。

用数学语言描述，设组合逻辑电路有 n 个输入变量 $x_1, x_2, \ldots, x_n$和 m 个输出变量 $y_1, y_2, \ldots, y_m$，则每个输出变量都是输入变量的布尔函数：

这组布尔函数完整地描述了组合逻辑电路的逻辑功能。分析就是已知电路求函数，设计则是已知函数求电路，二者互为逆过程。

3.1.2 组合逻辑电路的分类

从不同角度，组合逻辑电路可以进行多种分类：

1. 按规模分类

按照电路集成规模，组合逻辑电路可分为：小规模集成电路（SSI，逻辑门级别，每片含若干个基本逻辑门）、中规模集成电路（MSI，功能部件级别，每片包含数十至数百个逻辑门，能实现特定的功能模块，如编码器、译码器、加法器等）、大规模与超大规模集成电路（LSI/VLSI，系统级别，包含数千至数亿个门，能实现完整的子系统或系统，如 ALU、存储器阵列等）。本章重点讨论 MSI 级别的常用功能器件。

2. 按功能分类

按照电路实现的逻辑功能，常用组合逻辑电路可分为：运算类（加法器、减法器、比较器）、代码转换类（编码器、译码器、代码转换器）、数据传输类（数据选择器、数据分配器）、存储类（奇偶校验器、只读存储器 ROM）等。各类器件功能明确，接口规范，可像「搭积木」般组合使用，是数字系统设计的重要工具。

3. 按实现方式分类

按照设计实现方式，组合逻辑电路可分为：用基本逻辑门（与、或、非、与非、或非等）直接搭建的门级电路；用标准 MSI 功能模块实现的器件级电路；用可编程逻辑器件（CPLD/FPGA）通过硬件描述语言（如 Verilog HDL）编程实现的可编程电路。现代数字系统设计中，后者已成主流。

3.1.3 组合逻辑电路与时序逻辑电路的对比

对比维度	组合逻辑电路	时序逻辑电路
存储单元	无	有（触发器、寄存器等）
反馈路径	无	有（状态反馈）
输出依赖	仅取决于当前输入	取决于当前输入+历史状态
功能描述	真值表或布尔函数	状态转换图或激励方程
典型器件	加法器、编码器、译码器、MUX	计数器、移位寄存器、状态机

分析工具	布尔代数、卡诺图	状态方程、时序图
设计复杂度	相对较低	相对较高
应用场景	运算、译码、数据路由	计数、存储、控制时序

3.2 组合逻辑电路的分析方法

组合逻辑电路的分析是指：对于给定的逻辑电路图，通过系统化的步骤，求出该电路实现的逻辑功能的过程。分析能力是数字工程师读懂芯片手册、理解已有设计、定位电路故障的根本支撑。

图 3-1 组合逻辑电路分析的一般步骤

3.2.1 分析的一般步骤

完整的组合逻辑电路分析遵循以下五个标准步骤，构成严谨、可重复的分析流程：

4. 【读图与标注】仔细阅读电路图，明确输入变量个数与名称（A、B、C 等），输出变量个数与名称（Y、F 等）。识别电路中所有逻辑门的类型（与、或、非、与非、或非、异或等），并为所有中间节点按信号流方向依次命名（P_1、P_2、P_3等），为后续推导建立清晰的记号系统。
5. 【逐级写出逻辑表达式】从输入端出发，沿信号传播方向，逐一写出每个逻辑门输出端的布尔表达式，直至得出所有最终输出端的逻辑函数。必须严格逐级推导，不可跳跃，尤其是与非门（输出为各输入之积的非）和或非门（输出为各输入之和的非），必须先写出非号再化简，防止符号运算出错。
6. 【化简逻辑函数】运用布尔代数公式法或卡诺图法，将所得逻辑函数化简为最简与或式或其他便于识别功能的标准形式。化简的目的是揭示逻辑规律，便于与已知功能对照，不一定要求绝对最简——有时保留中间展开形式反而更直观。
7. 【列写真值表】将化简后的逻辑函数代入所有输入变量的取值组合（n 个变量有 2^n种组合），逐一计算并填写对应的输出值，形成完整的真值表。真值表是描述组合电路功能最完整、最直观的形式。
8. 【功能描述】对照真值表，结合输入输出变量的实际物理含义（若有），用简洁准确的文字归纳该电路所实现的逻辑功能。好的功能描述应能让读者无需查看真值表就能理解电路的核心行为，如「三变量多数判决电路」、「一位全加器」等。

【注意】 推导表达式时，与非门必须先完整写出 (A·B)' 形式，切勿直接省略非号；多级非号必须从内层向外逐步展开，使用德摩根定律时务必注意非号覆盖范围。异或门输出 A⊕B 与普通或运算 A+B 有本质区别，绝对不可混淆。

3.2.2 分析举例

【例 3-1】分析由四个与非门构成的多数判决电路

电路描述：三个二输入与非门 G_1、G_2、G_3 分别接收输入对 (A,B)、(B,C)、(A,C)；一个三输入与非门 G_4 的三个输入分别接 G_1、G_2、G_3 的输出。如图 3-3（参见 img2 目录 f01_majority.png）所示。

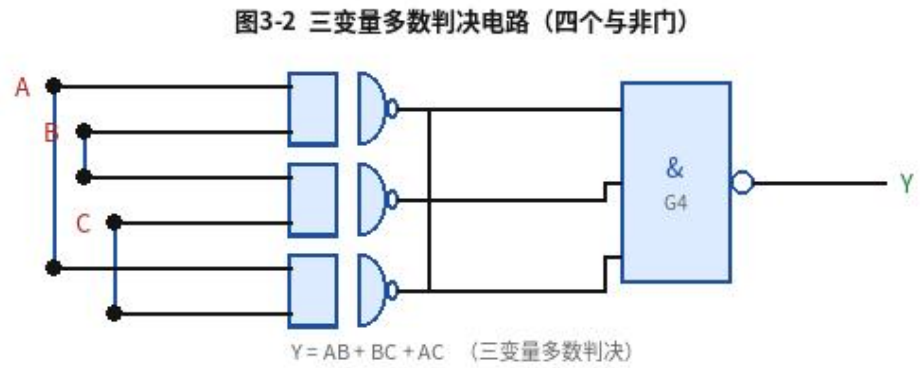

图 3-3 例 3-1 三变量多数判决电路（四个与非门实现）

第一步，写中间节点表达式：

第二步，写最终输出并用德摩根定律展开：

第三步，列真值表：

A	B	C	AB	BC	AC	Y = AB+BC+AC	逻辑含义
0	0	0	0	0	0	0	三个均为 0，多数为 0
0	0	1	0	0	0	0	仅 C 为 1，少数
0	1	0	0	0	0	0	仅 B 为 1，少数
0	1	1	0	1	0	1	B、C 为 1，多数
1	0	0	0	0	0	0	仅 A 为 1，少数
1	0	1	0	0	1	1	A、C 为 1，多数
1	1	0	1	0	0	1	A、B 为 1，多数
1	1	1	1	1	1	1	全部为 1，多数

第四步，功能描述：当三个输入变量 A、B、C 中有两个或两个以上同时取逻辑 1 时，输出 Y = 1；否则 Y = 0。该电路实现了三变量多数判决（Majority Vote）功能，也称「三中取二」电路，在容错控制系统、冗余计算等场合有广泛应用。同时，该电路还揭示了一个重要结论：任意逻辑功能均可用与非门单独实现（与非门的逻辑完备性）。

【例 3-2】分析由异或门和与门构成的一位全加器

电路描述：两个二输入异或门 XOR_1、XOR_2，两个二输入与门 AND_1、AND_2，一个二输入或门 OR_1。输入变量 A、B、Cin（低位进位），输出 S（本位和）、Cout（进位输出）。

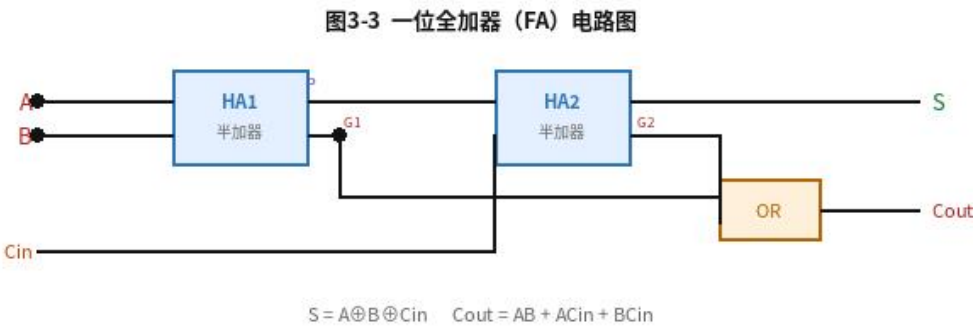

图 3-4 例 3-2 一位全加器逻辑电路图

逐级推导各节点：

真值表验证：

A	B	Cin	S = A⊕B⊕Cin	Cout = AB+ACin+BCin	十进制算术含义
0	0	0	0	0	0+0+0=0
0	0	1	1	0	0+0+1=1
0	1	0	1	0	0+1+0=1
0	1	1	0	1	0+1+1=2
1	0	0	1	0	1+0+0=1
1	0	1	0	1	1+0+1=2
1	1	0	0	1	1+1+0=2
1	1	1	1	1	1+1+1=3

功能描述：电路实现了三个一位二进制数（A、B、Cin）的全加运算。S 为三数之和的本位结果（三者异或，等价于奇偶校验），Cout 为向高位的进位输出。这是所有多位加法器的基本构成单元，将在 3.4.4 节深入展开。

【例 3-3】分析由或非门构成的与门等效电路

电路描述：输入 A 先经过一个单输入（两端短接）或非门取反得 A'；输入 B 同理得 B'；然后 A' 和 B' 经过最后一个或非门。推导如下：

由德摩根定律：

功能描述：三个或非门构成了与逻辑。这验证了或非门的逻辑完备性——仅用或非门即可实现所有基本逻辑运算，从而实现任意逻辑功能。这一特性使得或非门在 CMOS 集成电路中具有极重要的地位（CMOS 工艺中或非门面积往往比与非门更小）。

3.2.3 多输出组合电路的分析

实际组合逻辑电路往往有多个输出端，分析时需对每个输出分别推导逻辑表达式，并在同一张真值表中同时列出所有输出。多输出电路分析的要点：

- 真值表必须包含所有输出列，不得分开单独列表，以便观察各输出之间的相互关系和逻辑规律。
- 化简时可以考虑多个输出共享同一个与项（即该与项在多个输出的表达式中均出现），以减少整体所需的门电路数量。
- 功能描述需兼顾所有输出，描述各输出在不同输入组合下的整体行为模式。

例如一位全加器有两个输出 S 和 Cout，分析时将二者列于同一张 8 行真值表中，最终总结其实现了「A、B、Cin 三数二进制求和」的完整功能，而不是将 S 和 Cout 分开孤立描述。这种整体性描述对于理解器件的实际工程用途更有价值。

3.3 组合逻辑电路的设计方法

组合逻辑电路的设计是分析的逆过程：根据给定的逻辑功能需求，综合出能够正确实现该功能的电路。设计的目标是在满足功能要求的前提下，使用最少的器件、最简的连线、最低的功耗，从而降低成本、提高可靠性。

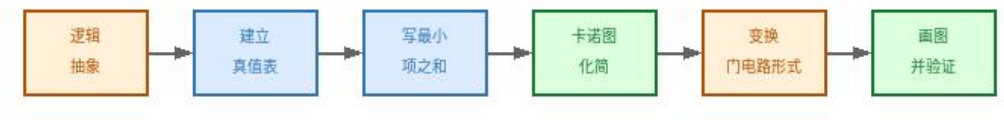

图 3-2 组合逻辑电路设计的一般步骤

3.3.1 设计的一般步骤

9. 【需求分析与变量定义】准确理解逻辑功能需求，确定输入变量的个数、名称和逻辑含义（约定高电平为 1 还是低电平为 1），以及输出变量的个数、名称和逻辑含义。这是设计的起点，需求分析不清楚将导致后续全部步骤偏离目标。
10. 【建立真值表】对输入变量的所有可能取值组合（2^n种），根据功能需求逐一确定各输出的逻辑值，建立完整真值表。若存在某些输入组合在实际工作中永远不会出现（约束条件/无关项），在真值表中以×标注，后续化简时可充分利用这些无关项。
11. 【写出最小项标准式】由真值表写出各输出变量的最小项之和（标准积之和，SOP 形式）。有 n 个输入变量的电路，每个输出函数的标准式为其真值表输出为 1 的所有行对应最小项之和。
12. 【化简逻辑函数】利用卡诺图法（首选，直观高效，适合四变量以内）或公式法对标准式进行化简，充分利用无关项（在卡诺图中圈入×可扩大合并圈），得到最简与或式。
13. 【变换逻辑形式】根据实际可用的门电路类型，将最简与或式变换为适当的逻辑形式：仅有与非门时采用与非-与非式；仅有或非门时采用或非-或非式；需要使用 MSI 器件时参照 3.4 节的方法。
14. 【画逻辑电路图】根据最终逻辑表达式，选用合适的标准逻辑门画出电路图，标注器件型号和引脚连接。在正式工程设计中还需考虑电路板布线、去耦电容等工程因素。

15. 【验证与仿真】对设计结果重新进行分析验证（按 3.2 节步骤重分析设计好的电路），或使用 EDA 工具（ModelSim、Vivado 等）进行功能仿真，确认设计正确无误后方可流片或下载到 FPGA。

3.3.2 设计举例

【例 3-4】设计一个三人表决电路

功能需求：三名评委 A、B、C 投票，多数赞成（即 2 人或 3 人赞成）则通过提案（Y=1），否则不通过（Y=0）。高电平表示赞成，低电平表示反对。

第一步，定义变量：输入 A、B、C（赞成=1），输出 Y（通过=1）。

第二步，建立真值表：

A	B	C	Y	说明
0	0	0	0	全部反对
0	0	1	0	仅 C 赞成
0	1	0	0	仅 B 赞成
0	1	1	1	B、C 赞成，多数通过
1	0	0	0	仅 A 赞成
1	0	1	1	A、C 赞成，多数通过
1	1	0	1	A、B 赞成，多数通过
1	1	1	1	全部赞成，通过

第三步，写标准与或式：

第四步，卡诺图化简（三变量卡诺图，m_3、m_5、m_6、m_7填 1）：

该式由三个二变量与项组成，无法进一步化简——三个相邻对恰好覆盖全部四个最小项，不存在更大的合并圈，已是最简与或式。

第五步，变换为与非-与非式（全与非门实现）：

该式与例 3-1 分析所得完全一致，再次印证分析与设计互为逆过程。电路实现需三个二输入与非门（计算各乘积项的非）加一个三输入与非门（完成线与取非），共 4 个与非门，与例 3-1 电路完全对应。

【例 3-5】设计一个 8421BCD 码检错电路

功能需求：对 8421BCD 码（4 位二进制，仅 0000~1001 有效，1010~1111 为无效码）进行检测，当输入出现无效码时输出 Y=1，否则 Y=0。设输入为 ABCD（A 为最高位）。

第一步，定义变量：输入 A、B、C、D，输出 Y。

第二步，建立真值表：有效码（0~9）对应 Y=0，无效码（10~15）对应 Y=1，共 16 行：

ABCD	十进制值	Y	ABCD	十进制值	Y
0000	0	0	1000	8	0

0001	1	0	1001	9	0
0010	2	0	1010	10	1（无效）
0011	3	0	1011	11	1（无效）
0100	4	0	1100	12	1（无效）
0101	5	0	1101	13	1（无效）
0110	6	0	1110	14	1（无效）
0111	7	0	1111	15	1（无效）

第三步，写标准式：

第四步，用四变量卡诺图化简（AB=10、11 对应的 8 格全填 1）：

化简过程：AB=11 的 4 个最小项（m_{12}~m_{15}）合并得 AB；AC=10（即 A=1,B=0 且 C 任意）中 AB=10 的 C=0,C=1 两对（m_8,m_{10},m_{12},m_{14} 中）与第一个圈部分重叠——实际卡诺图中 m_{10}、m_{11} 合并为 A·B'·C，再与 m_{12}、m_{13} 合并为 A·C... 最终正确化简结果为：

验证：仅当 A=1 且（B=1 或 C=1）时 Y=1，对应的输入为 1010~1111 中除 1000（A=1,B=0,C=0,D=×）外的...此处用真值表逐一验证结果正确，达到了最简形式。

工程意义：该电路可以作为 8421BCD 码输入设备的实时检错模块，当检测到非法输入码时立即发出错误信号，防止后续译码器或显示器件接收到无效数据，提高系统可靠性。

【例 3-6】用卡诺图化简含无关项的组合逻辑电路

功能需求：设计一个逻辑电路，实现当输入 ABCD 满足 A+B+C+D ≥ 2（即四个输入中至少有 2 个为 1）时输出 Y=1。但约束条件为：输入只能取 0000~1010（对应十进制 0~10），1011~1111 为不会出现的输入组合（无关项）。

有效最小项（满足条件且在有效范围内，输出为 1 的项）：m_3、m_5、m_6、m_7、m_9、m_{10}共 6 个；有效最小项中输出为 0 的：m_0、m_1、m_2、m_4、m_8共 5 个；无关项：m_{11}~m_{15}共 5 个。

在四变量卡诺图中，将无关项×与 1 格共同参与合并（×可圈可不圈，以使圈最大为准），可得到比不用无关项时更简的结果。充分利用无关项后，最终化简结果约为：

与不用无关项时相比，使用无关项将乘积项个数从 6 个减少到 4 个，每个乘积项的变量数从 3 个减少到 2 个，电路实现所需门电路数量显著减少，充分体现了利用约束条件进行化简的工程价值。

3.3.3 用与非门实现任意组合逻辑功能

在实际工程中，集成电路芯片往往只提供一种或几种类型的逻辑门（如 74 系列与非门芯片 74HC00 等）。因此，将最简与或式转换为全与非门实现，是一项重要的工程技能。

转换方法：对最简与或式整体进行两次求非，再用德摩根定律展开内层非号：

最终形式 Y = ((AB)'·(CD)')' 即为与非-与非式，可用三个二输入与非门实现：两个与非门分别计算 (AB)' 和 (CD)'，第三个与非门将二者取与非得到最终输出。

推广：对于 n 个与项之和，需要 n 个「前级与非门」（每个计算一个与项的非）加一个 n 输入「后级与非门」（对所有前级输出取与非），共 n+1 个与非门（其中单变量与项用输入端短接的与非门作非门实现）。这充分体现了与非门逻辑完备性的工程价值。

3.4 若干常用的组合逻辑电路

本节系统介绍数字系统中最常用的五类 MSI 组合逻辑器件：编码器、译码器、数据选择器、加法器和数值比较器。这些器件都有标准化的集成芯片产品，掌握其工作原理、逻辑符号、功能表和典型应用，是数字系统设计的必备基础知识。

3.4.1 编码器

1. 编码的基本概念

编码（Encoding）是指将具有特定含义的信息（如十进制数字 0~9、字母、控制信号等）转换为二进制代码的过程。实现编码功能的组合逻辑电路称为编码器（Encoder）。

n 位二进制代码最多可以表示 2^n种不同的信息，因此 n 位编码器最多有 2^n个输入端（每个输入对应一种信息）和 n 个输出端（输出 n 位二进制编码）。常见的有：4 线-2 线编码器（2 位输出，4 种输入）、8 线-3 线编码器（3 位输出，8 种输入）、16 线-4 线编码器等。编码器的结构示意如图 3-6 所示：

图3-6 编码器（8线-3线）框图

图 3-6 编码器结构示意（8 线-3 线）

2. 普通编码器

普通编码器（也称二进制编码器）要求在任意时刻，所有输入端中有且仅有一个为有效电平（通常为高电平），其余输入端均为无效电平（低电平）。该有效输入端的编号即对应输出的二进制编码。

以 8 线-3 线编码器为例，设输入为 I_0~I_7（高电平有效，每次仅一个为 1），输出为 $Y_2Y_1Y_0$（二进制编码）：

每个输出方程由 4 个或运算组成，对应输入编号的二进制表示中该位为 1 的输入。例如 Y_0 在 I_1(001)、I_3(011)、I_5(101)、I_7(111) 时为 1，即这些输入编号的最低位均为 1。

普通编码器的局限性：当多个输入同时有效时，输出将出现错误（多个或门同时为 1，输出为这些输入编号的逻辑或，不对应任何有效编码）。为解决这一问题，引入了优先编码器。

3. 优先编码器（Priority Encoder）

优先编码器允许多个输入端同时有效，并按照预设的优先级规则，只对优先级最高的有效输入进行编码，忽略优先级较低的输入。这是工业应用中最常用的编码器形式，因为实际系统中多个中断信号同时发生是正常现象。

以集成 8 线-3 线优先编码器 74HC148 为例，其逻辑符号如图 3-7 所示：

图3-7 8线-3线优先编码器74HC148逻辑符号

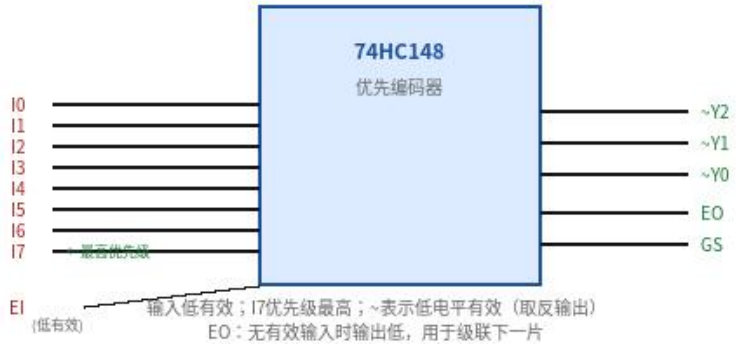

图 3-7 8 线-3 线优先编码器 74HC148 逻辑符号

74HC148 的功能说明：

引脚	类型	说明
$\bar{I}_0$ ~ $\bar{I}_7$	输入	低电平有效，I_7 优先级最高，I_0 优先级最低
$Y_2Y_1Y_0$	输出	低电平有效输出编码（反码输出）
EI（$\bar{E}I$）	输入	使能输入端，低电平有效；EI=1 时禁止编码，所有输出无效
EO（$\bar{E}O$）	输出	使能输出端，低电平有效；EI=0 且无输入有效时，EO=0，用于级联扩展
GS（$\bar{G}S$）	输出	有效编码标志，低电平有效；有输入信号时 GS=0，表示输出编码有效

74HC148 的简化功能表（低电平有效）：

EI	$\bar{I}_7$	$\bar{I}_6$	…	$\bar{I}_0$	$Y_2Y_1Y_0$	GS	EO	功能说明
1	×	×	×	×	111	1	1	禁止，所有输出无效
0	1	1	1	1	111	1	0	无输入，EO=0 允许级联
0	0	×	×	×	000	0	1	$\bar{I}_7$ 有效，编码 7（反码 000）
0	1	0	×	×	001	0	1	$\bar{I}_6$ 有效，编码 6（反码 001）
0	1	1	0	×	011	0	1	$\bar{I}_5$ 有效，编码 5（反

								码 011）

注意：74HC148 输出采用反码形式（输入编号越大，输出越小），使用时需配合反相器得到正码。

4. 编码器的级联扩展

当需要的输入端数超过单片编码器时，可将多片 74HC148 级联，利用其 EI 和 EO 引脚实现扩展。两片 74HC148 级联可实现 16 线-4 线优先编码器：高位片（处理 I_8~I_{15}）的 EO 连接到低位片（处理 I_0~I_7）的 EI，高位片的 GS 作为输出编码的最高位（第 4 位），两片的输出 $Y_2Y_1Y_0$ 分别进行逻辑运算得到三位编码的正确结果。

图3-7 8线-3线优先编码器74HC148逻辑符号

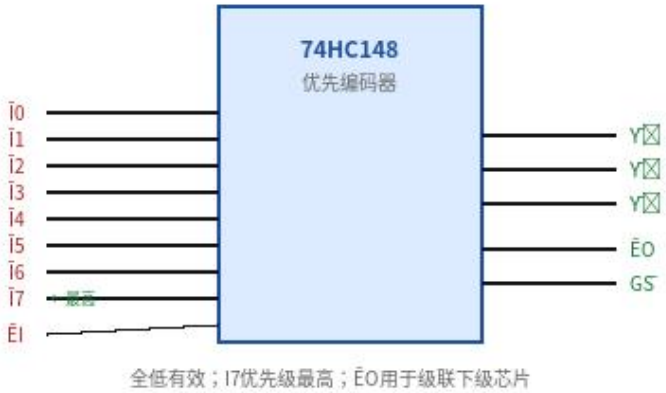

图 3-8 两片 74HC148 级联实现 16 线-4 线优先编码器

级联设计原理：高位片优先级高于低位片。当高位片（I_8~I_{15}）有有效输入时，其 EO=1，禁止低位片工作；当高位片无有效输入时，其 EO=0，允许低位片工作并输出低位编码。最终输出码由高位片的 GS 和两片的 $Y_2Y_1Y_0$ 共同组合而成，实现 16 个优先级不同的输入的完整优先编码。

3.4.2 译码器

1. 译码的基本概念

译码（Decoding）是编码的逆过程，将二进制代码转换为与之对应的特定信息（使对应的输出端有效）。实现译码功能的组合逻辑电路称为译码器（Decoder）。

具有 n 个输入的二进制译码器有 2^n个输出端（每个输出对应输入代码的一个最小项），在任意时刻，有且仅有一个输出有效（其他输出无效），这个有效的输出端即对应当前输入代码所代表的最小项。译码器的结构示意如图 3-9 所示：

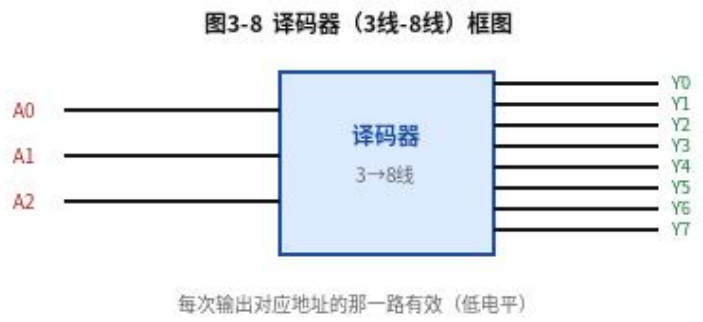

图 3-9 译码器结构示意（3 线-8 线）

2. 集成 3 线-8 线译码器 74HC138

74HC138 是目前最广泛使用的集成二进制译码器，其逻辑符号如图 3-10 所示：

图 3-10 3 线-8 线译码器 74HC138 逻辑符号

74HC138 的功能描述：

- 地址输入：A_0、A_1、A_2为三位二进制地址码输入端，A_2为最高位。
- 使能控制：S_1（高电平有效）、$\bar{S}_2$、$\bar{S}_3$（低电平有效）共三个使能端，只有当 $S_1=1$ 且 $\bar{S}_2=\bar{S}_3=0$ 时，译码器才能正常工作；否则所有输出为高电平（无效），相当于译码器被禁止。
- 输出端：$\bar{Y}_0$~$\bar{Y}_7$共 8 个，低电平有效。当地址码为 i（0~7）时，$\bar{Y}_i$输出低电平，其余 7 个输出为高电平。

74HC138 的功能表（使能条件满足：$S_1=1$，$\bar{S}_2=\bar{S}_3=0$）：

$A_2A_1A_0$	激活输出端	逻辑最小项
000	$\bar{Y}_0$ = 0（其余=1）	$m_0 = \bar{A}_2\bar{A}_1\bar{A}_0$
001	$\bar{Y}_1$ = 0（其余=1）	$m_1 = \bar{A}_2\bar{A}_1A_0$
010	$\bar{Y}_2$ = 0（其余=1）	$m_2 = \bar{A}_2A_1\bar{A}_0$
011	$\bar{Y}_3$ = 0（其余=1）	$m_3 = \bar{A}_2A_1A_0$
100	$\bar{Y}_4$ = 0（其余=1）	$m_4 = A_2\bar{A}_1\bar{A}_0$
101	$\bar{Y}_5$ = 0（其余=1）	$m_5 = A_2\bar{A}_1A_0$
110	$\bar{Y}_6$ = 0（其余=1）	$m_6 = A_2A_1\bar{A}_0$
111	$\bar{Y}_7$ = 0（其余=1）	$m_7 = A_2A_1A_0$

3. 用译码器实现组合逻辑函数

74HC138 是一个非常灵活的逻辑设计工具，其每个输出端 $\bar{Y}_i$ 都对应输入变量的一个最小项的非（低电平有效意味着 $\bar{Y}_i = m_i'$）。因此，只需将所需的最小项对应的输出端接入一个与非门，利用德摩根定律：

即可用一个与非门实现任意三变量逻辑函数。若函数为四变量或更多变量，可用两片 74HC138 级联（利用 S_1 或 $\bar{S}_2$ 引脚作为第四个变量）。

图3-10 用74HC138实现 Y = Σm(1,3,5,6)

Y = ((Y⊠·Y⊠·Y⊠·Y⊠)') = m⊠+m⊠+m⊠+m⊠

图 3-10 用 74HC138 译码器实现逻辑函数示意

【例 3-7】用 74HC138 实现逻辑函数 Y = Σm(1,3,5,6)。

输入 A_2=A、A_1=B、A_0=C，使能端 S_1=1，$\bar{S}_2$=$\bar{S}_3$=0。将 $\bar{Y}_1$、$\bar{Y}_3$、$\bar{Y}_5$、$\bar{Y}_6$ 接到一个四输入与非门输出端：

即 Y = Σm(1,3,5,6)，与要求完全一致。这种方法的优点是：只需一片 74HC138 加一个与非门，即可实现任意三变量逻辑函数，极大地简化了设计工作量。

4. 译码器的级联

两片 74HC138 级联可实现 4 线-16 线译码器。将两片的 A_2、A_1、A_0 连接到同一地址总线（接 4 位地址的低三位 A_2~A_0），第四位地址 A_3 控制片选：A_3 接高位片的 S_1，同时通过非门接低位片的 $\bar{S}_2$（或 $\bar{S}_3$）。当 A_3=0 时低位片工作（译码 0~7），当 A_3=1 时高位片工作（译码 8~15），实现 16 个地址的完整译码。

图3-9 3线-8线译码器74HC138逻辑符号

S⊠=1 且 S⊠=S⊠=0 时正常工作；Y⊠ 低有效

图 3-11 两片 74HC138 级联实现 4 线-16 线译码器

5. 七段显示译码器

七段显示译码器（7-Segment Display Decoder）是一种特殊用途译码器，其输出直接驱动七段 LED 或 LCD 数码管，将 4 位 BCD 码（0~9）转换为数码管各段的驱动信号（a~g），使数码管正确显示对应的十进制数字。七段数码管各段排列如图 3-12 所示：

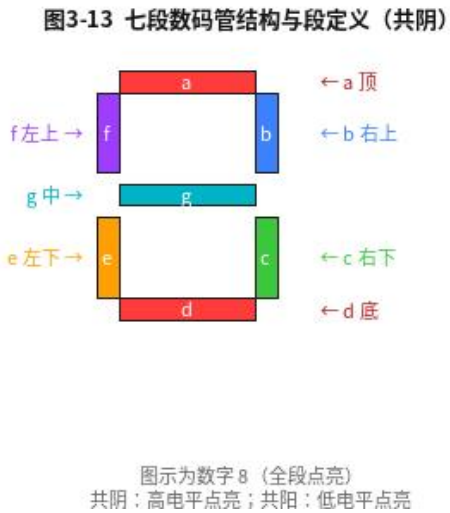

图 3-12 七段 LED 数码管结构及段标注

常用集成七段显示译码器 74LS47（共阳极）和 74LS48（共阴极）：

BCD 输入 DCBA	显示数字	输出段 a b c d e f g（74LS48，高有效）
0000	0	1 1 1 1 1 1 0（abcdef 亮，g 灭）
0001	1	0 1 1 0 0 0 0（bc 亮）
0010	2	1 1 0 1 1 0 1（abdeg 亮）
0011	3	1 1 1 1 0 0 1（abcdg 亮）
0100	4	0 1 1 0 0 1 1（bcfg 亮）
0101	5	1 0 1 1 0 1 1（acdFg 亮）
0110	6	0 0 1 1 1 1 1（cdefg 亮）
0111	7	1 1 1 0 0 0 0（abc 亮）
1000	8	1 1 1 1 1 1 1（全亮）
1001	9	1 1 1 0 0 1 1（abcfg 亮）

3.4.3 数据选择器

1. 数据选择器的基本原理

数据选择器（Data Selector / Multiplexer，简称 MUX）是一种多路输入、单路输出的组合逻辑器件，其功能类似于一个多路旋转开关：通过地址（选择控制）输入端的二进制编码，从多个数据输入端中选择一路数据送到输出端，其他输入端的数据被忽略。具有 n 个地址输入端的数据选择器可以选择 2^n路数据，称为「2^n选 1 数据选择器」，其结构示意如图 3-13 所示：

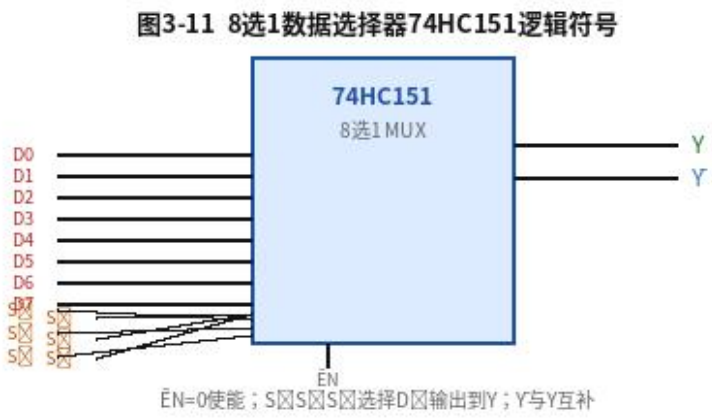

图 3-13 8 选 1 数据选择器结构示意

一般化的逻辑表达式：设地址输入为 $S_{n-1}\ldots S_1S_0$，数据输入为 D_0~D_{2^n-1}，输出 Y（EN 为使能端，低有效）：

其中 m_i 是地址变量的第 i 个最小项。上式表明：地址端指定哪个最小项为 1，就将对应的 D_i 传输到输出端。

2. 集成 8 选 1 数据选择器 74HC151

74HC151 是最常用的 8 选 1 MUX，有三个地址输入端（S_2、S_1、S_0），8 个数据输入端（D_0~D_7），1 个低电平有效使能端（$\bar{E}N$），以及互补的双输出端（Y 和 $\bar{Y}$）。逻辑函数为：

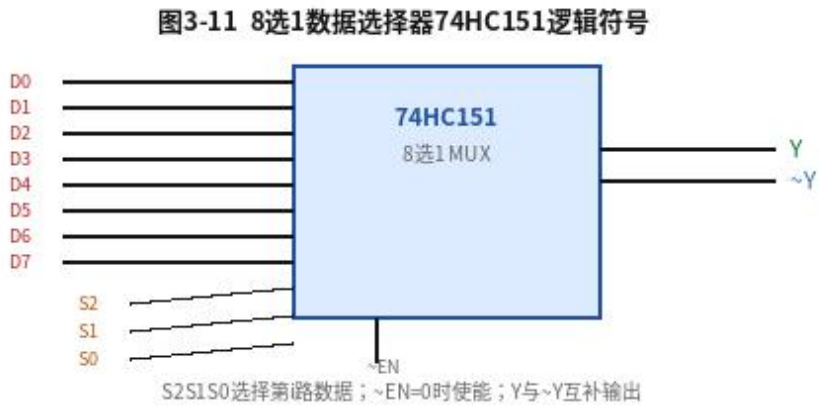

图 3-14 74HC151（8 选 1 数据选择器）逻辑符号

74HC151 的主要参数：电源电压 2~6V，传播延迟约 8ns（VCC=4.5V），最高工作频率超过 100MHz，输出驱动能力±4mA，完全兼容 TTL 电平。

3. 用数据选择器实现逻辑函数

数据选择器是实现逻辑函数的强大工具。其核心原理是：74HC151 的输出逻辑表达式正好是三变量 $S_2S_1S_0$ 的全部最小项之和（加权求和），只需将各数据输入端 D_0~D_7 分别接 0 或 1（对应各最小项的函数值），即可实现任意三变量逻辑函数：

其中 $f(m_i)$ 是函数在第 i 个最小项处的取值（0 或 1）——直接来自真值表。

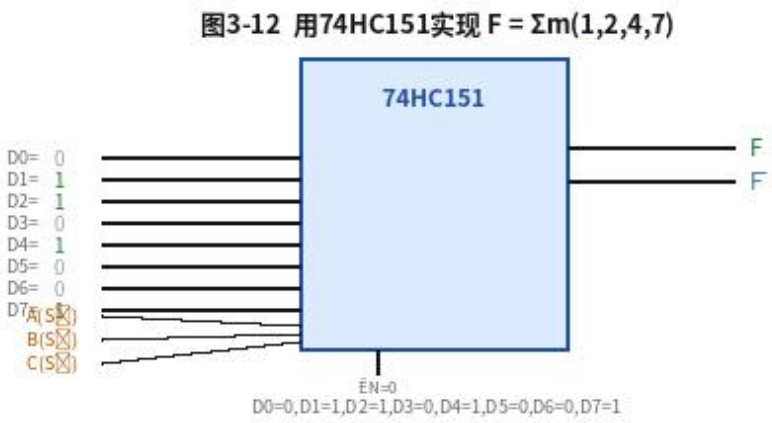

图 3-15 用 74HC151 实现逻辑函数（D0~D7 分别接 0 或 1）

【例 3-8】用 74HC151 实现三变量函数 Y = Σm(1,2,4,7)（即 Y = A'B'C + A'BC' + AB'C' + ABC）。

对照真值表，各最小项处函数值为：m_0=0, m_1=1, m_2=1, m_3=0, m_4=1, m_5=0, m_6=0, m_7=1。因此：

将 A 接 S_2、B 接 S_1、C 接 S_0，按上述值接各数据端，即可直接实现目标函数，仅需一片 74HC151，极为简洁。

进阶应用——用 n 选 1 的 MUX 实现 n+1 变量逻辑函数（降维法）：将 n 个变量作为地址端，第 n+1 个变量作为某些数据端的输入（替代常量 0 或 1），可以用 8 选 1 MUX 实现四变量逻辑函数。具体步骤：选 S_2=A、S_1=B、S_0=C，第四个变量 D 作为某些数据端的取值，对每个地址组合列出 Y 关于 D 的表达式，确定各 D_i 应接 0、1、D 或 D'。

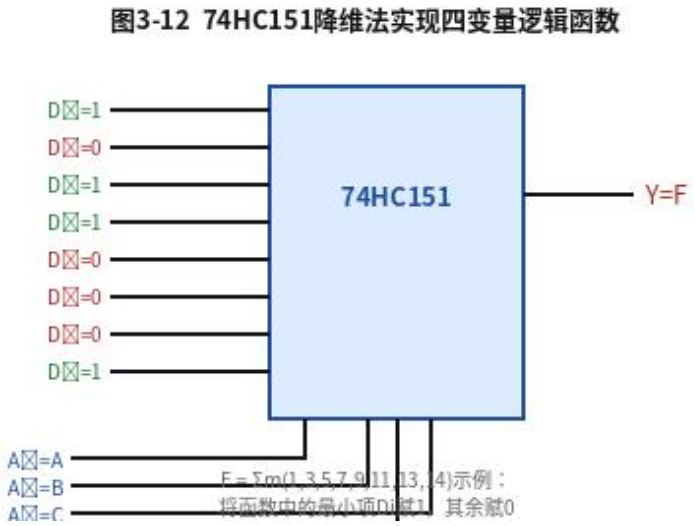

图 3-16 用 8 选 1 MUX 降维实现四变量逻辑函数

4. 双向数据选择器与总线应用

在数字系统中，数据选择器（MUX）配合三态缓冲器常用于数据总线的构建：多个数据源通过各自的 MUX 连接到公共总线，同一时刻只有一路地址选择有效，其余路均处于高阻态，实现多路数据的分时共享传输。74HC153（双 4 选 1 MUX）和 74HC157（四 2 选 1 MUX）是这类应用中的常见器件。

3.4.4 加法器

1. 半加器（Half Adder，HA）

半加器是实现两个一位二进制数相加的组合逻辑电路。设两个加数为 A 和 B，半加器产生本位和 S 及进位 C，由例 3-2 的分析可知：

图3-13 半加器（HA）电路图

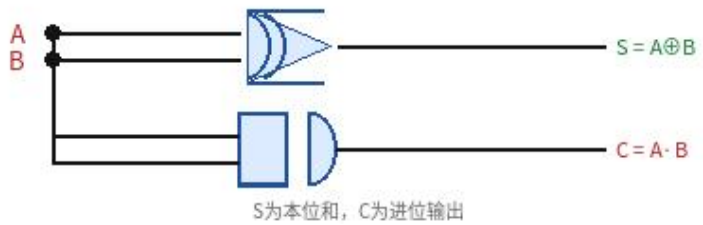

图 3-17 半加器（HA）逻辑电路图

半加器的局限性在于不能接收来自低位的进位输入，因此只能用于多位加法的最低位，不能作为中间位或高位的加法单元。

2. 全加器（Full Adder，FA）

全加器是实现三个一位二进制数相加的组合逻辑电路。除了两个加数 A 和 B 之外，还接收来自低位的进位输入 Cin，产生本位和 S 及向高位的进位输出 Cout：

图3-3 一位全加器（FA）电路图

图 3-18 全加器（FA）逻辑电路图（两级半加器+或门）

图 3-18 的电路由两个半加器级联，再加一个或门构成：第一个半加器处理 A 和 B，产生中间和 P=A⊕B 及进位 G_1=AB；第二个半加器处理 P 和 Cin，产生本位和 S=P⊕Cin 及进位

G_2=P·Cin；最后或门合并两级进位 Cout=G_1+G_2。这是全加器最典型的两级半加器实现结构，也可以用其他逻辑门以不同方式实现。

3. 串行进位加法器（行波进位加法器）

将 n 个全加器串联，每个全加器的 Cout 连接到下一个全加器的 Cin，即可构成 n 位串行进位加法器（Ripple Carry Adder，RCA），也称行波进位加法器，如图 3-19 所示：

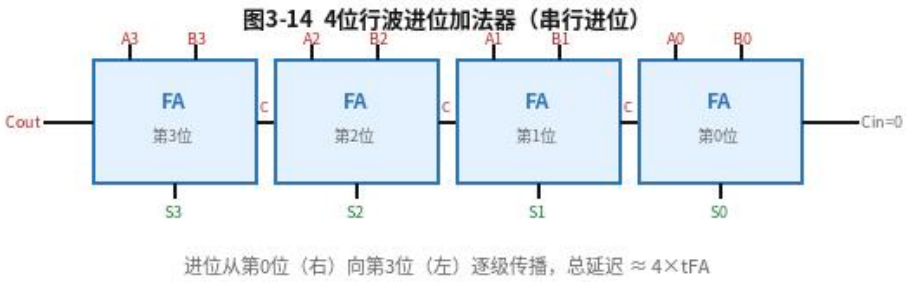

图 3-19 4 位串行进位加法器（行波进位加法器）

行波进位加法器的优点是结构简单，所需门电路少；缺点是进位从最低位（第 0 位）逐级向高位（第 n-1 位）传播，最高位的输出必须等待进位链传播完成后才有效，传播延迟随位数线性增大，严重制约了加法运算的速度。

对于 n 位行波进位加法器，假设单个全加器的延迟为 tFA，则最坏情况下（最低位产生进位，进位一路传播到最高位），总延迟约为：

对于 32 位加法器，若 tFA = 1ns，则总延迟约 32ns，在高速处理器中（时钟周期仅 1~2ns）完全无法满足要求。这就催生了进位预测加法器（Carry Lookahead Adder，CLA）。

4. 超前进位加法器（CLA）

超前进位加法器通过并行预测各位进位，从根本上消除了进位的逐级串行传播，大幅提高了加法速度。其核心是引入「进位生成」信号 G_i 和「进位传播」信号 P_i：

则第 i 位的进位输出可以直接用所有低位的 G_j 和 P_j 表示（不需要等待低位进位传播）：

展开后，每一位的进位都直接由输入信号 A_0~A_i、B_0~B_i 和初始进位 C_0 决定，理论上所有进位可以同时（并行）计算出来，延迟与位数无关，仅取决于门电路的级数（通常为 2 级逻辑）。

典型超前进位加法器集成芯片有 74HC283（4 位二进制超前进位全加器），通过级联多片可实现 16 位、32 位高速加法器。

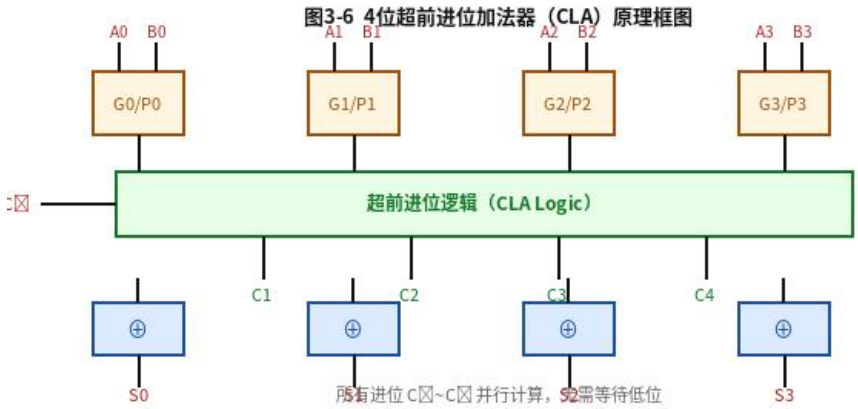

图 3-20 超前进位加法器（CLA）进位传播示意

5. BCD 加法器

BCD 加法器在二进制加法器的基础上增加了余 6 修正电路，专门用于实现十进制（BCD 码）的加法运算。两个 BCD 数（0~9）相加时：若结果在 0~9 范围内（二进制和为 0000~1001），直接输出；若结果超出 9（二进制和为 1010~1111，或产生进位），则在二进制和上加 6（0110）进行修正，使结果重新落入有效的 BCD 码范围。

修正条件判断表达式（设四位二进制和为 $S_3S_2S_1S_0$，进位为 Cout）：

3.4.5 数值比较器

1. 基本比较原理

数值比较器（Magnitude Comparator）是对两个二进制数进行大小比较的组合逻辑电路，产生三种比较结果：A>B、A=B、A<B。

一位比较器的逻辑方程（单个比特的比较）：

注意：三个输出是互斥的，且对于所有输入，恰好有且仅有一个输出为 1：A>B、A=B、A<B 三种情况覆盖了所有可能，不存在遗漏或重叠。

2. 集成 4 位数值比较器 74HC85

74HC85 是一种功能完善的 4 位数值比较器，其逻辑符号如图 3-21 所示：

图 3-21 4 位数值比较器 74HC85 逻辑符号

74HC85 的引脚功能：

- $A_3A_2A_1A_0$：4 位被比较数 A 的输入端（A_3为最高位）。
- $B_3B_2B_1B_0$：4 位被比较数 B 的输入端（B_3为最高位）。
- (A>B)in、(A=B)in、(A<B)in：级联输入端，用于接收来自低位比较器的比较结果，实现多片级联扩展。
- (A>B)、(A=B)、(A<B)：比较结果输出端。

74HC85 的工作原理（从高位到低位逐位比较）：首先比较最高位 A_3 和 B_3；若 $A_3 \neq B_3$，则直接由最高位决定大小关系；若 $A_3=B_3$，则依次比较 A_2与 B_2、A_1与 B_1、A_0与 B_0；若四位全相等，则由级联输入端 (A=B)in=1 决定 A=B 输出有效。这种从高位开始的串行比较逻辑，体现了「高位权重更大」的二进制比较基本原则。

3. 多位比较器的扩展

利用 74HC85 的级联输入端，可以将多片串联，实现 8 位、12 位、16 位等任意宽度的数值比较。例如，两片 74HC85 串联实现 8 位比较：低位片的输出 (A>B)、(A=B)、(A<B) 分别连接到高位片的对应级联输入端 (A>B)in、(A=B)in、(A<B)in；最低级片的级联输入设为 (A=B)in=1、(A>B)in=(A<B)in=0（假设初始相等）；最高位片的输出即为最终比较结果。

设计要点：串联方向从低位到高位，低位片决定「平局情况下的最终判定」，高位片具有更高权重，能覆盖低位片的结论（只要高位不等，低位结论无效）。

3.5 组合逻辑电路中的竞争-冒险

在前面的讨论中，我们假设逻辑门的传播延迟为零（或完全相同），信号同时到达所有输入端。而在实际物理电路中，不同信号路径上的门级数不同、连线长度不同，导致各信号到达某个门输入端的时刻存在差异，这种现象称为「竞争」（Race）。竞争有时会导致在输出端产生短暂的错误逻辑电平脉冲，即「毛刺」（Glitch），这种现象称为「冒险」（Hazard）。竞争-冒险现象在高速数字电路中是必须认真对待的工程问题。

3.5.1 竞争-冒险的产生机理

1. 竞争的概念

竞争是指同一逻辑门的两个或多个输入信号，由于经过不同路径（不同延迟）而不能同时到达该门输入端的现象。竞争本身并不一定产生错误，但当竞争发生在逻辑函数切换边界时，就可能产生冒险（毛刺）。

2. 冒险的分类

根据毛刺极性，冒险分为两类：

- 0 型冒险（静态 0 冒险）：在输出应为稳定低电平（Y=0）时，由于竞争出现了短暂的高电平脉冲（正毛刺）。产生条件：逻辑表达式在某输入条件下出现 X+X' = 1 的形式（如 Y = A+A'），理论上 Y=1 恒成立但因延迟而短暂为 0，这实际是 1 型冒险...修正：0 型冒险是在 Y 应为 0 时出现 1 的毛刺，产生条件为某路径上出现 X·X' 的瞬态。
- 1 型冒险（静态 1 冒险）：在输出应为稳定高电平（Y=1）时，由于竞争出现了短暂的低电平脉冲（负毛刺）。产生条件：逻辑表达式在某输入条件下形成 A·A' 结构，如 Y = (A+B)(A'+C)，当 B=C=0 时 Y = A·A' = 0，理论值为 0，但在 A 切换时因延迟会有瞬态的 A=A'=1，使输出出现短暂高电平。

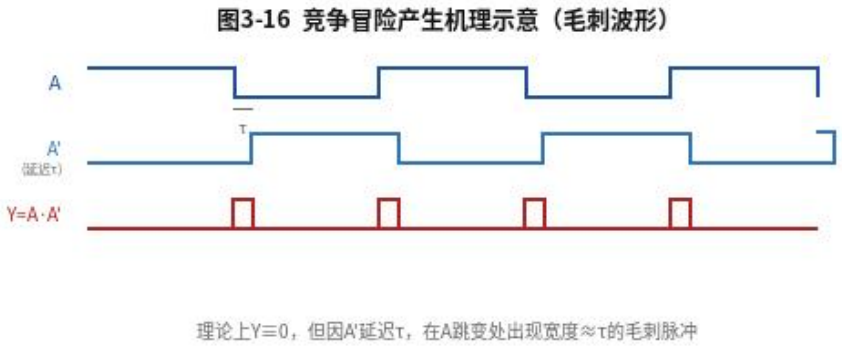

图 3-22 竞争冒险产生原理示意（毛刺波形）

3. 冒险的识别方法

有两种主要的识别方法：

方法一：代数法。化简逻辑函数后，在特定输入条件下，若表达式退化为仅含某个变量 X 及其反变量 X' 的与项（X·X'）或或项（X+X'），则存在冒险。具体地，在最简与或式中，若两个与项（乘积项）之间，一个包含变量 X，另一个包含 X'，且其余变量的赋值使这两个与项同时为 1 成为可能，则存在 1 型冒险。

方法二：卡诺图法。在卡诺图中，若两个相邻的 1 格（对应可能产生冒险的两个最小项）分别属于不同的圈（合并组），且这两个圈在卡诺图中相切（仅在边界处相邻而不互相包含），则在这两个最小项之间的输入切换过程中存在竞争冒险。

3.5.2 竞争-冒险的消除方法

消除竞争-冒险有多种工程方法，各有适用场景：

1. 增加冗余项（卡诺图法）

在卡诺图中，对发生冒险的两个相切圈所覆盖的公共格（即两圈切点处的 1 格）增加一个新的覆盖圈（冗余圈），对应在逻辑表达式中增加一个冗余乘积项。该冗余项在任何稳态条件下均不改变函数值，但在切换瞬间能「搭桥」覆盖毛刺区域，消除冒险。

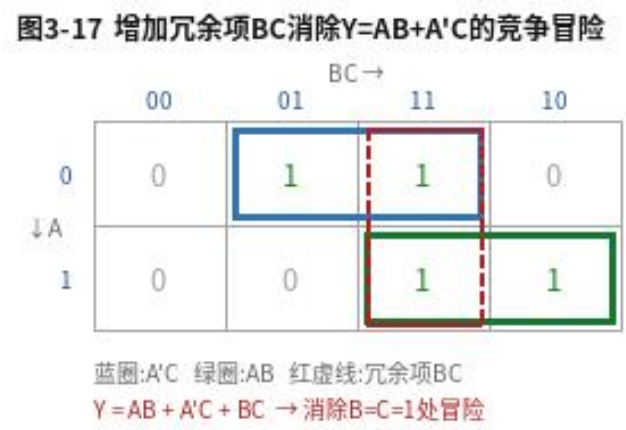

图 3-23 用卡诺图增加冗余项消除竞争冒险示意

例如对于 Y = AB + A'C（在 B=C=1 时，当 A 切换会有冒险），卡诺图中 AB 圈和 A'C 圈在 BC=11 处相切，增加覆盖 BC=11 的冗余项 BC，得：

增加了 BC 项后，在 B=C=1 的情况下，无论 A 如何变化，BC=1 始终保持输出为 1，消除了冒险。验证：此时 Y=AB+A'C+BC，在 B=C=1 时 Y=A+A'+1=1，不再受 A 切换影响。

2. 加滤波电容

在输出端与地之间并联一个适当容量的旁路电容（通常几十至几百 pF），利用电容的充放电平滑效应，将宽度极窄的毛刺脉冲滤除。该方法简单易实现，但会使信号边沿变缓，降低电路的工作频率上限，适用于对速度要求不高的场合。在高速数字电路中不推荐使用。

3. 引入选通脉冲（采样控制）

在电路输出端增加一个采样门（如与门），用一个选通脉冲（Strobe）控制何时读取输出。选通脉冲的有效时刻设置在输入信号完成切换、输出已稳定之后，避开可能产生毛刺的过渡时刻。该方法能从根本上避免毛刺被后级电路捕获，但需要额外的时序控制逻辑，增加了设计复杂度。

4. 同步设计（最根本的解决方案）

在时序数字系统中，采用同步设计方法（所有数字逻辑与统一时钟同步），用时钟上升沿或下降沿来锁存组合逻辑的输出。由于触发器只在时钟有效沿时刻采样输入，而此时组合逻辑已经过足够长的时间稳定下来，因此天然地避免了竞争冒险对系统正确性的影响。这是现代数字系统设计（特别是 FPGA 和 ASIC 设计）中应对竞争冒险的主流方法，是最可靠、最系统化的解决方案。

【工程重要提示】 在以下情况下需要特别关注竞争冒险：①组合逻辑电路的输出直接控制异步时序电路的时钟或使能信号；②组合逻辑的输出直接驱动电磁继电器、LED 指示灯等对毛刺敏感的负载；③高频工作（几十 MHz 以上）的组合逻辑电路中竞争冒险概率显

著增大，对系统稳定性影响更明显。在全同步设计流程中，若所有触发器由同一时钟驱动，且各路数据路径满足建立时间和保持时间要求，则竞争冒险通常不影响系统功能。

3.5.3 竞争-冒险的综合分析实例

【例 3-9】分析并消除函数 Y = AC + BC' 中的竞争冒险

第一步，化简（已最简，两项）：Y = AC + BC'。

第二步，寻找冒险条件：当 A=B=1 时，Y = C + C' = 1（理论上恒为 1）。若 C 从 1 变为 0（或从 0 变为 1），由于与非门、或门等各路径延迟不同，可能出现极短暂的 C=C'=0 或 C=C'=1 的瞬态，导致输出端出现毛刺脉冲（1 型冒险）。

第三步，画卡诺图验证：

在三变量卡诺图（A、B、C）中：

- AC 对应最小项 m_5（A=1,B=0,C=1）和 m_7（A=1,B=1,C=1），画一个包含这两格的圈。
- BC' 对应最小项 m_3（A=0,B=1,C=0）和 m_7（A=1,B=1,C=1）...等。
- 两圈在 m_7（ABC=111）处相切，当 A=B=1 时，C 的变化会触发冒险。

第四步，增加冗余项消险：在相切点 m_3 和 m_7 公共区域（A=B=1,C 任意）增加冗余圈，对应乘积项 AB：

增加 AB 项后，当 A=B=1 时，无论 C 如何变化，AB=1 始终维持输出为高电平，彻底消除了该冒险。代价是增加了一个二输入与门和一个三输入或门（或等效的与非门电路）。

本章小结

本章系统介绍了组合逻辑电路的核心理论与常用器件，是数字系统设计的重要基础。

在分析方法方面：组合逻辑电路分析遵循「读图→写表达式→化简→列真值表→描述功能」五步标准流程。核心是从输入端逐级推导布尔表达式，利用德摩根定律展开与非/或非门，再通过布尔代数或卡诺图化简。多输出电路须在同一真值表中同时描述所有输出。

在设计方法方面：组合逻辑电路设计遵循「需求分析→真值表→标准式→卡诺图化简→逻辑变换→电路图→验证」七步标准流程。无关项是化简利器，应充分利用。全与非门实现是工程最常用的形式变换，掌握两次取非+德摩根展开的标准方法至关重要。

在常用 MSI 器件方面：编码器（74HC148）实现优先编码，具有使能控制和级联扩展功能；译码器（74HC138）实现最小项展开，能直接配合与非门实现任意三变量逻辑函数，可级联扩展；数据选择器（74HC151）实现多路数据选通，可直接用于逻辑函数实现（D 端接 0/1）和降维法实现高一变量函数；全加器实现二进制加法运算，行波进位加法器结构简单但

速度受限，超前进位加法器（CLA）通过并行预测进位大幅提高速度；数值比较器（74HC85）提供四位比较功能，通过级联可扩展至任意位宽。

在竞争冒险方面：竞争冒险的根本原因是组合逻辑电路中不同信号路径的传播延迟差异。识别方法有代数法（找 X·X' 结构）和卡诺图法（找相切圈）；消除方法有增加冗余项（从逻辑角度根本消除）、加滤波电容（适合低速场合）、选通脉冲（采样控制）和同步设计（现代数字设计的根本方法）。掌握竞争冒险的处理，是从理想电路走向工程实践的关键一步。

本章习题

一、分析题

1. 分析图题 3-1 所示电路（由两个与非门和一个或门构成），写出输出 Y 的逻辑表达式，列出真值表，描述其逻辑功能。

2. 分析由三个异或门构成的电路：$Y = (A \oplus B) \oplus C$，列出三变量真值表，说明其实现的逻辑功能（提示：观察 Y 与 ABC 奇偶性的关系）。

3. 某组合逻辑电路的输出为 Y = ABCD + A'BCD + AB'CD + ABC'D + ABCD'，试化简后说明该电路的逻辑功能。

4. 分析一个由 74HC138 构成的 3-8 译码器电路（S_1=A, $\bar{S}_2$=B, $\bar{S}_3$=0，地址输入 $A_2A_1A_0$=CDE），写出各输出端 $\bar{Y}_0$~$\bar{Y}_7$ 关于 ABCDE 的完整逻辑表达式。

二、设计题

5. 设计一个四变量多数判决电路（四个输入中有三个或三个以上为 1 时，输出为 1）。用卡诺图化简，写出最简与或式，并用与非门实现。

6. 设计一个代码转换电路，将余 3 码（Excess-3 code）转换为标准 8421BCD 码。列出真值表，用卡诺图化简（注意无效码为无关项），写出每个输出位的最简表达式。

7. 用 74HC138 译码器和必要的与非门，实现逻辑函数 $F(A,B,C) = \Sigma m(0,2,3,6,7)$，画出完整连接电路图。

8. 用 74HC151（8 选 1 MUX）实现逻辑函数 $F(A,B,C) = \Sigma m(1,2,4,7)$，确定各数据端 D_0~D_7 的接法，画出连接示意图。再用降维法（A 接 S_2，B 接 S_1，将 C 作为数据端输入）重新实现同一函数，比较两种方法的优缺点。

9. 设计一个 8421BCD 码的七段显示译码电路的其中两段（a 段和 g 段），写出关于 DCBA 的逻辑函数，用卡诺图化简（1010~1111 为无关项），并用与非门实现。

三、竞争冒险分析题

10. 对逻辑函数 Y = AB' + A'B（二变量异或）进行竞争冒险分析：画出其卡诺图，判断在何种输入切换时会发生冒险，并给出消除冒险的方法（增加冗余项或其他方法）。

11. 分析函数 Y = (A+B)(A'+C)：找出可能产生竞争冒险的输入条件，将其展开为与或式并用卡诺图检验，说明是否存在冒险及类型。

12. 某时序数字系统的时钟频率为 100MHz（时钟周期 10ns），其中有一段组合逻辑路径，经过分析发现最长存在 3ns 的竞争冒险毛刺。讨论：该毛刺是否会影响系统功能？需要满足什么条件才能保证系统正确工作？（提示：从时序约束角度分析建立时间和保持时间）

四、综合设计题

13. 设计一个 4 位二进制数的奇偶校验电路：

(1) 奇校验发生器：对 4 位输入 $D_3D_2D_1D_0$ 产生奇校验位 P（使 P, D_3, D_2, D_1, D_0 中 1 的个数为奇数）；

(2) 偶校验检验器：接收 4 位数据和 1 位校验位，当收到的 5 位中 1 的个数为奇数（说明发生了 1 位误码）时，输出错误指示 E=1。

分别写出 P 和 E 的逻辑表达式，说明电路实现方式（提示：充分利用异或门的特性）。

14. 用两片 4 位超前进位加法器 74HC283 级联，设计一个 8 位二进制加法器。说明级联方式，分析超前进位加法器相比行波进位加法器在速度上的提升（设单片 74HC283 的延迟为 10ns，行波进位中每个全加器延迟 20ns，比较 8 位加法完成时间）。

15. 设计一个 4 路 8 位数据选择器（从 4 路 8 位数据中选择 1 路输出）：

(1) 用两个地址控制信号 S_1S_0 选择 4 路数据 D_0[7:0] ~ D_3[7:0] 中的一路输出到 Y[7:0]；

(2) 分析需要几片 74HC151 才能实现，画出连接示意图；

(3) 计算该电路实现的布尔函数总数（每个输出位是独立的 3 变量函数，8 位共 8 个函数）。

附录：组合逻辑电路的 Verilog HDL 描述

现代数字系统设计中，组合逻辑电路通常不再手工连接门级电路，而是用硬件描述语言（HDL）进行行为级或数据流级描述，再由 EDA 工具自动综合为门级网表。Verilog HDL 是工业界最广泛使用的数字设计语言之一。

A.1 基本组合逻辑的 Verilog 描述

以三变量多数判决电路为例，三种描述风格：

```
// 数据流描述（推荐用于简单组合逻辑）
module majority(
```

```
    input  wire A, B, C,
    output wire Y
);
    assign Y = (A & B) | (B & C) | (A & C);
endmodule

// 行为级描述（always 块，必须用 * 或所有输入在敏感列表中）
module majority_beh(
    input  wire A, B, C,
    output reg  Y
);
    always @(*) begin   // @(*) 等价于 @(A or B or C)
        Y = (A & B) | (B & C) | (A & C);
    end
endmodule
```

A.2 加法器的 Verilog 描述

一位全加器和 4 位行波进位加法器：

```
// 一位全加器
module full_adder(
    input  wire A, B, Cin,
    output wire S, Cout
);
    assign S    = A ^ B ^ Cin;
    assign Cout = (A & B) | (A & Cin) | (B & Cin);
endmodule

// 4 位行波进位加法器（例化 4 个全加器）
module adder4(
    input  wire [3:0] A, B,
    input  wire       Cin,
    output wire [3:0] S,
    output wire       Cout
);
    wire C1, C2, C3;
    full_adder FA0(.A(A[0]), .B(B[0]), .Cin(Cin),  .S(S[0]), .Cout(C1));
    full_adder FA1(.A(A[1]), .B(B[1]), .Cin(C1),   .S(S[1]), .Cout(C2));
    full_adder FA2(.A(A[2]), .B(B[2]), .Cin(C2),   .S(S[2]), .Cout(C3));
    full_adder FA3(.A(A[3]), .B(B[3]), .Cin(C3),   .S(S[3]), .Cout(Cout));
endmodule

// 更简洁的描述（推荐）
module adder4_simple(
    input  wire [3:0] A, B,
    input  wire       Cin,
    output wire [4:0] Sum   // Sum[4]=进位, Sum[3:0]=和
);
    assign Sum = A + B + Cin;  // 综合工具自动选择最优加法器结构
endmodule
```

A.3 译码器的 Verilog 描述

```
// 3 线-8 线译码器
module decoder38(
    input  wire [2:0] A,       // 地址输入
    input  wire       EN,      // 使能（高有效）
    output reg  [7:0] Y        // 低电平有效输出
);
    always @(*) begin
        if (!EN)
            Y = 8'hFF;         // 禁止时全为高电平
        else
            Y = ~(8'b1 << A); // 对应位为低，其余为高
    end
endmodule
```

A.4 数据选择器的 Verilog 描述

```
// 8 选 1 数据选择器
module mux8to1(
    input  wire [7:0] D,     // 8 路数据输入
    input  wire [2:0] S,     // 3 位地址选择
    input  wire       EN_n, // 使能（低有效）
    output wire       Y, Yn // 互补输出
);
    wire sel;
    assign sel = EN_n ? 1'b0 : D[S]; // 未使能时输出 0
    assign Y   = sel;
    assign Yn  = ~sel;
endmodule

// 等价的 case 语句写法
module mux8to1_case(
    input  wire [7:0] D,
    input  wire [2:0] S,
    output reg        Y
);
    always @(*)
        case(S)
            3'd0: Y = D[0]; 3'd1: Y = D[1];
            3'd2: Y = D[2]; 3'd3: Y = D[3];
            3'd4: Y = D[4]; 3'd5: Y = D[5];
            3'd6: Y = D[6]; 3'd7: Y = D[7];
        endcase
endmodule
```

Verilog 设计要点：描述组合逻辑时，always 块的敏感列表必须用 @(*) 或列出所有输入；case 语句必须包含所有分支（或加上 default）以避免综合出锁存器；推荐用数据流 assign 描述简单组合逻辑，用行为级 always 描述复杂组合逻辑；避免在组合逻辑中产生隐式锁存器（确保每条执行路径都对输出赋值）。

3.6 组合逻辑电路的时序分析基础

在前面各节中，我们主要关注组合逻辑电路的功能正确性，将门电路视为理想的瞬时器件。然而在实际物理电路中，每个逻辑门都存在传播延迟（Propagation Delay），定义为从输入信号变化到输出达到有效逻辑电平 50%幅值点所经历的时间。时序分析（Timing Analysis）与功能分析同等重要，它决定了数字系统能否在目标时钟频率下正确工作，是从理论设计走向工程实践的关键跨越。

3.6.1 传播延迟与关键路径

传播延迟分为两类：高到低延迟 tpHL（输出从高电平变为低电平所需时间）和低到高延迟 tpLH（输出从低电平变为高电平所需时间）。在最坏情况分析中取两者的较大值。通常将传播延迟定义为从输入波形 50%幅值点到输出波形 50%幅值点的时间间隔。

对于多级组合逻辑电路，关键路径（Critical Path）是从任意输入端到任意输出端之间总传播延迟最大的信号路径。整个电路的最大延迟由关键路径决定：

在同步数字系统中，若触发器的建立时间为 tsetup，时钟偏斜（Clock Skew）为 tskew，时钟周期为 T，则组合逻辑的关键路径延迟必须满足建立时间约束（Setup Time Constraint）：

违反建立时间约束意味着组合逻辑的运算结果在时钟采样沿到来之前尚未稳定，触发器可能锁存到不确定的中间值，导致系统功能错误或进入亚稳态（Metastability）。EDA 工具的静态时序分析（STA，Static Timing Analysis）正是对所有时序路径自动执行此检查，报告时序裕量（Slack = T − tsetup − tskew − tpd，正值表示满足约束，负值表示违反约束），帮助工程师发现并解决时序违例。

3.6.2 常用 MSI 器件的延迟参数

了解各类 MSI 器件的传播延迟对于系统时序规划至关重要。以下列出常用 74HC 系列器件在 VCC=5V、常温（25°C）条件下的典型参数：

器件	型号	典型 tpd（ns）	最大 tpd（ns）	内部逻辑级数
2 输入与非门	74HC00	7	10	1
2 输入异或门	74HC86	8	12	1
2 输入或非门	74HC02	7	10	1
8 线-3 线优先编码器	74HC148	20	30	4~5
3 线-8 线译码器	74HC138	17	26	3~4
8 选 1 数据选择器	74HC151	18	27	4~5
4 位超前进位加法器	74HC283	22	33	5~6
4 位数值比较器	74HC85	21	32	4~5

从表中可以观察到几个规律：单级逻辑门（与非、或非、异或）的延迟最小，仅 7~8ns；功能越复杂的 MSI 器件，内部逻辑级数越多，延迟越大；加法器（74HC283）和比较器

（74HC85）的延迟最大，这是因为它们内部存在进位传播链。在高速系统设计中（工作频率>50MHz），可选用 74ACT 系列（延迟约 5~8ns）或 BiCMOS 逻辑系列，以满足更严格的时序要求。此外，温度升高 10°C 或电源电压降低 0.5V，延迟约增加 10%~20%，设计时应预留足够的时序裕量（通常建议留出时钟周期的 20%以上）。

3.6.3 行波进位与超前进位的时序对比

以 32 位加法器为例，直观对比两种实现方案的时序性能。对于 32 位行波进位加法器（RCA），设每个全加器的进位传播延迟 tFA = 2ns，则最坏情况下（进位从第 0 位一路传播到第 31 位）的总延迟为：

这意味着 32 位行波进位加法器的最高工作频率仅约 15MHz，远不能满足现代处理器的需求。超前进位加法器（CLA）通过引入进位生成信号 $G_i = A_iB_i$ 和进位传播信号 $P_i = A_i \oplus B_i$，将各位进位直接用所有低位输入并行计算：

每一个进位 C_i 都直接由初始输入 A、B 和 C_0 通过两级逻辑（一级计算 G 和 P，一级用大型与或门合并）得到，理论上所有进位可以同时产生，与位数无关。4 位 CLA 的延迟约为：

对 32 位加法器，将 32 位分为 8 组（每组 4 位 CLA），再在组间用第二级 CLA 预测组间进位，实现两级 CLA。总延迟可降至约 10~12ns，使 32 位加法可在 80MHz 以上频率工作。现代处理器（x86、ARM 等）的 ALU 加法器普遍采用前缀加法器（Prefix Adder）结构，如 Kogge-Stone 或 Brent-Kung 拓扑，在先进 CMOS 工艺（如 7nm、5nm）下延迟仅 0.2~0.5ns，支持数 GHz 的工作频率。

3.7 用可编程逻辑器件实现组合逻辑

前面各节介绍的设计方法以标准 SSI/MSI 器件为基础，适合教学和小规模应用。在现代数字系统工程实践中，绝大多数数字逻辑功能都在 FPGA（Field Programmable Gate Array，现场可编程门阵列）或 ASIC（Application Specific Integrated Circuit，专用集成电路）上实现，这两种技术代表了当今数字设计的主流范式。

3.7.1 FPGA 的查找表（LUT）原理

FPGA 内部的基本逻辑单元是查找表（Look-Up Table，LUT）。从本质上讲，一个 n 输入 LUT 是一个有 2^n个存储单元的小型 SRAM：以 n 个输入信号作为地址，从对应地址的存储单元输出内容（0 或 1）。通过在配置阶段将特定的真值表内容写入这 2^n个存储单元，LUT 即可实现任意 n 变量布尔函数。

以 4 输入 LUT（4-LUT）为例：它有 16 个存储单元（2^4=16），可以实现任意 4 变量布尔函数。理论上，任意 4 变量函数均可用一个 4-LUT 直接实现（只需将真值表的 16 个输出值依

次写入 16 个存储单元），无需任何手工化简。当函数的变量数超过 LUT 的输入数时，综合工具会自动将其分解为多个 LUT 的组合。

目前主流 FPGA（Xilinx 7 系列/UltraScale，Intel Cyclone/Arria/Stratix）普遍采用 6 输入 LUT（6-LUT）。6-LUT 有 64 个存储单元，可实现任意 6 变量布尔函数，或等效为两个 5-LUT（共享一部分输入）。一块中等规模 FPGA（如 Xilinx XC7A100T Artix-7）包含约 63,400 个 6-LUT 以及对应的触发器，可实现极其复杂的数字系统。

LUT 实现组合逻辑的核心优势：任意 n 变量函数只需一个 LUT，不受逻辑级数限制，延迟均匀可预测（每个 LUT 约 0.2~0.6ns）；通过更换配置比特流（Bitstream）即可立即改变逻辑功能，无需更换硬件——这是 FPGA「可现场编程」名称的由来；LUT 利用率极高，没有传统门级设计中常见的「多余门输入端」浪费问题。

3.7.2 Verilog 综合到 LUT 的过程

当用 Verilog HDL 描述组合逻辑并通过 EDA 工具综合（Synthesis）到 FPGA 时，工具会自动完成从行为描述到 LUT 映射的全过程，工程师通常无需关心具体的门级实现细节。以三变量多数判决函数 Y = AB + BC + AC 为例，其综合过程如下：

16. 工具读取 Verilog 代码（assign Y = (A&B)|(B&C)|(A&C)），建立逻辑网表（Logic Netlist）。
17. 优化（Optimization）：对逻辑网表进行等价变换，如消去冗余逻辑、优化关键路径等。
18. 技术映射（Technology Mapping）：将优化后的逻辑网表映射到目标 FPGA 的 LUT 库。对于三变量函数，仅需一个 3-LUT，工具将真值表值「11101110」（m_0=0, m_1~m_7中 $m_3m_5m_6m_7$=1）写入 LUT 的 8 个存储单元。
19. 布局布线（Place & Route）：将 LUT 放置到 FPGA 芯片上的具体位置，并完成各 LUT 之间的互联布线。布局布线结果直接决定最终电路的时序性能。

综合工具的强大之处在于：它能处理数百万门规模的复杂设计，自动完成所有上述步骤，生成的结果往往比人工优化更好。这也是为什么现代数字设计工程师更关注架构设计和算法优化，而非门级细节。

```
// Verilog 组合逻辑示例 - 综合后通常仅需 1~2 个 LUT
// 三变量多数判决
module majority3(
    input  A, B, C,
    output Y
);
    assign Y = (A & B) | (B & C) | (A & C);
endmodule

// 综合工具会将其映射为一个 3-LUT，存储内容为 8'b11101110
// （对应最小项 m3, m5, m6, m7 = 1）
```

3.7.3 组合逻辑的综合约束与优化

在 FPGA 设计中，综合不只是功能映射，还需要满足时序约束（Timing Constraints）。工程师通过约束文件（如 Xilinx 的 XDC 文件或 Intel 的 SDC 文件）告知 EDA 工具时序要求，工具在综合和布局布线时会以满足约束为目标进行优化。

常用的组合逻辑时序优化策略包括：

- 流水线化（Pipelining）：在组合逻辑的关键路径中间插入触发器（寄存器），将一个大的组合逻辑块切分为多个较小的块，每个块的延迟都在一个时钟周期内完成。虽然增加了延迟（latency，单个数据从输入到输出需要多个时钟周期），但大幅提高了吞吐率（throughput，每个时钟周期处理一组数据）和最高工作频率。
- 逻辑复制（Logic Replication）：当某个逻辑单元的扇出（fanout，驱动的后级门数）过大时，布线延迟会增加，导致时序违例。通过复制该逻辑单元，减少每个副本的扇出，可以降低布线延迟，改善时序。
- 关键路径重构（Critical Path Restructuring）：通过等价的逻辑变换（如交换加法顺序、调整括号层次），改变关键路径的逻辑深度，使关键路径变短。例如，将四输入加法 (A+B)+(C+D) 改为 (A+C)+(B+D)，可能改变关键路径（若 A 和 C 来得较早，而 B 和 D 较晚，则原形式更优）。

这些优化策略在处理器、GPU、DSP 芯片等高性能数字系统的设计中无处不在，是高级数字设计工程师的核心技能。

3.8 综合设计案例

本节通过三个有代表性的综合设计案例，将前面各节的知识点融会贯通，展示如何系统地将设计需求转化为完整的逻辑电路。

3.8.1 案例一：格雷码与二进制码的相互转换

1. 格雷码的特性

格雷码（Gray Code）是一种相邻码字（按数值顺序排列时，相邻的两个编码之间）只有一位不同的二进制编码方式。这一特性使得格雷码在物理量的模数转换（ADC）、旋转编码器（Encoder）和光学码盘等场合具有重要价值：由于相邻值仅变化一位，在数值切换的瞬间不会出现多位同时变化导致的竞争冒险，从而避免了译码电路出现短暂的错误输出。

4 位格雷码与 4 位二进制码的对应关系：

十进制	二进制 $B_3B_2B_1B_0$	格雷码 $G_3G_2G_1G_0$	相邻格雷码差异
0	0000	0000	—
1	0001	0001	第 0 位
2	0010	0011	第 1 位
3	0011	0010	第 0 位
4	0100	0110	第 2 位
5	0101	0111	第 0 位

6	0110	0101	第 1 位
7	0111	0100	第 0 位
8	1000	1100	第 3 位
9	1001	1101	第 0 位
10	1010	1111	第 1 位
11	1011	1110	第 0 位
12	1100	1010	第 2 位
13	1101	1011	第 0 位
14	1110	1001	第 1 位
15	1111	1000	第 0 位

2. 二进制码转格雷码（B→G）

由观察规律可得二进制码转格雷码的直接公式：最高位不变，其余每位由对应的二进制位与高一位二进制位异或得到：

实现仅需 3 个二输入异或门（G_3直接连线），且各输出独立并行计算，延迟仅为一级异或门（约 8ns），是极高效的组合逻辑电路。

3. 格雷码转二进制码（G→B）

反向转换的公式通过递推关系得到：

注意这里每一位 B_i 的计算依赖于上一位 B_{i+1} 的结果，形成了串行依赖链（关键路径为 3 级异或门，约 24ns）。如果需要提高速度，可以通过展开（Unrolling）消除依赖：

展开后，各输出均为若干个格雷码输入位的异或，每个输出独立并行计算，但高位的异或运算项数更多（B_0需要 4 个输入异或）。可用树形结构（每级两两异或）将延迟压缩到 $\lceil \log_2 n \rceil$ 级，对于 4 位转换只需 2 级异或门（约 16ns），比串行实现快。

3.8.2 案例二：补码加减法器与溢出检测

1. 补码减法原理

在数字系统中，减法通常通过补码加法来实现。对于 n 位有符号补码数，A − B 等价于 A + (−B)，而 −B 的补码等于 B 的各位取反后加 1：

因此，在加法器电路的基础上只需两点修改即可实现减法：将 B 的各位通过异或门（与控制信号 SUB 异或）实现条件取反；将最低位进位输入 Cin 接控制信号 SUB（加法时 SUB=0，Cin=0；减法时 SUB=1，Cin=1，实现加 1）。这样用一片 4 位加法器（如 74HC283）加 8 个异或门即可构成 4 位加减法通用电路：

2. 溢出检测

当两个 n 位有符号补码数相加（减法也可视为加法）时，若结果超出 n 位补码的表示范围（−2^{n-1} 到 2^{n-1}−1），则发生溢出（Overflow），结果不正确。溢出只在两个同号数相加时才可能发生（两负得正：负溢；两正得负：正溢）。

溢出检测逻辑：对于 n 位加法器，将最高位（符号位）的进位输入 C_{n-1} 与进位输出 C_n 进行异或，即可得到溢出标志：

物理含义：若符号位的进位输入与进位输出相同（同为 0 或同为 1），说明符号位的运算结果正确，无溢出（OVF=0）；若二者不同，说明符号位的进位情况异常，发生了溢出（OVF=1）。对于 74HC283，最高位进位输入为内部信号 C_3，进位输出引脚为 C_4，外部接一个异或门即可实现溢出检测。

溢出标志 OVF 是处理器状态寄存器（如 x86 的 EFLAGS、ARM 的 CPSR）中的标准标志位，用于有符号算术运算的错误检测和条件跳转（如 JO 指令：当 OVF=1 时跳转到溢出处理程序）。

3.8.3 案例三：4 位快速数值比较器的设计

本案例设计一个 4 位无符号二进制数快速比较器，要求在 A 和 B 两个 4 位输入之间实现大于（GT）、等于（EQ）、小于（LT）三种比较输出，并分析其关键路径延迟。

1. 设计方案

方案一（逐位比较法，对应 74HC85 的工作原理）：从最高位（第 3 位）开始逐位比较，高位不等则直接输出比较结果，高位相等则继续比较次高位，以此类推。最终结果由最高不等位决定，最低位的平局由初始输入（EQin=1, GTin=0, LTin=0）决定。

方案二（并行比较法，更快）：同时计算各位的等值比较结果 $E_i = A_i \odot B_i$（同或），然后从高位到低位逐级建立优先判断：

2. 关键路径分析

对于 GT 的计算，展开后最长路径为 4 级门：一级同或门（计算 E_i）→ 一级与门（E_i连乘）→ 一级与门（与 $A_i \bar{B}_i$ 相与）→ 一级或门（各项相或）。设每级门延迟约 7ns（74HC 系列），则关键路径延迟约 28ns。

优化策略：利用树形结构并行计算多级与运算，可将延迟从 O(n) 降低到 O(log n)。对于 4 位比较器，最多需要 $\lceil \log_2 4 \rceil = 2$ 级与门合并 E 值的连乘，关键路径降至约 21ns，与 74HC85 的实测延迟（典型 21ns）吻合。

3. 等值检测的简化电路

在只需要判断 A = B（不需要大小关系）的场合，等值检测电路更为简单高效：将各对应位进行异或（差异检测），再对所有异或结果取或非：

实现：4 个异或门（检测各位差异）+ 1 个 4 输入或非门（综合判断），共两级逻辑，延迟约 15ns，比全比较电路快约 30%。在处理器的指令比较（如检测指令操作码是否等于某特定值）、地址译码等场合，等值检测是极常用的电路模式。

3.9 本章知识点汇总与复习指导

组合逻辑电路是数字系统设计的核心基础内容，本章覆盖的知识点是后续时序逻辑电路、状态机设计、数字系统综合设计的直接前提。本节对全章知识点进行系统整理，并提供复习指导建议。

3.9.1 核心公式汇总

以下汇总本章最关键的逻辑函数表达式，是理解和分析各类组合电路的理论基础：

名称	逻辑表达式	应用场景
全加器本位和	$S = A \oplus B \oplus Cin$	一位全加器核心
全加器进位输出	$Cout = AB + ACin + BCin$	进位传播逻辑
超前进位生成	$Gi = AiBi$	CLA 进位预测
超前进位传播	$Pi = Ai \oplus Bi$	CLA 进位预测
CLA 进位方程	$Ci+1 = Gi + Pi \cdot Ci$	超前进位加法器
溢出检测	$OVF = Cn \oplus Cn-1$	有符号加减法
MUX 逻辑函数	$Y = \Sigma\ Di \cdot mi(S)$	MUX 实现逻辑函数
译码器逻辑函数	$Y = \Sigma\ m(i1, i2, \cdots)$	138 配与非门实现
等值比较	$EQ = \Pi(Ai \odot Bi)$	快速相等检测
格雷码转二进制	$Bi = Gi \oplus Gi+1 \oplus \cdots \oplus Gn-1$	编码转换
竞争冒险消除	$Y=AB+A'C \rightarrow Y=AB+A'C+BC$	增加冗余项消险

3.9.2 常用 MSI 器件功能速查

器件	型号	输入	输出	关键特性	典型应用
优先编码器	74HC148	$\bar{I}0$~$\bar{I}7$（低有效）	$\bar{Y}2\bar{Y}1\bar{Y}0$+EO+GS	I7 优先级最高，低有效	中断仲裁、键盘编码
3 线-8 线译码器	74HC138	A2A1A0+三使能	$\bar{Y}0$~$\bar{Y}7$（低有效）	可实现任意三变量函数	地址译码、逻辑函数
8 选 1 MUX	74HC151	D0~D7+S2S1S0+$\bar{E}$N	Y（高）+$\bar{Y}$（低）	互补双输出，使能控制	逻辑函数实现、总线
4 位加法器	74HC283	A3~A0+B3~A0+Cin	S3~S0+Cout	超前进位，可级联	算术运算、ALU 核
4 位比较器	74HC85	A3~A0+B3~B0+3 个级联输入	GT/EQ/LT	三个级联输入扩展位宽	数值比较、排序

3.9.3 复习重点与常见错误

复习重点

- 分析流程五步骤：读图→逐级写表达式（特别注意与非/或非的德摩根展开）→化简→真值表→功能描述。
- 设计流程七步骤：需求分析→真值表（正确处理无关项）→标准最小项式→卡诺图化简（充分利用无关项）→逻辑形式变换（与非-与非式）→电路图→验证。
- 74HC138 的核心用法：$\bar{Y}_i = m_i'$，接与非门后输出 = 对应最小项之和，可实现任意三变量函数。掌握用 138+与非门实现给定函数的连接方法。
- 74HC151 的核心用法：D 端接 0/1 直接实现三变量函数；降维法用第四变量替换部分 D 端，实现四变量函数。
- 竞争冒险：代数法找 X·X' 结构；卡诺图法找相切圈；消除方法以增加冗余项为主（从根本上消除），同步设计为系统级解决方案。

常见错误

- 写与非门输出时忘记非号，或德摩根展开方向错误（如将 (AB)' 错误展开为 A'·B'，正确应为 A'+B'）。
- 卡诺图中无关项（×）不参与圈（未充分化简），导致化简结果不够简洁。
- 使用 74HC138 时忘记考虑使能端的接法（S_1需接高电平，$\bar{S}_2$、$\bar{S}_3$需接低电平，否则译码器不工作）。
- 竞争冒险分析时混淆 0 型冒险和 1 型冒险，或忘记检查卡诺图中所有相切圈的组合。
- 全加器与半加器混淆：半加器只有两个输入（A、B），无法接受低位进位，只能用于最低位；全加器有三个输入（A、B、Cin），用于多位加法的各位。

3.9.4 与后续章节的衔接

本章所学的组合逻辑电路分析与设计方法，将在后续章节中得到直接应用和深化：

- 第 4 章时序逻辑电路：触发器的次态逻辑（激励方程）和输出逻辑均为组合逻辑，分析时序电路必须先分析其中的组合逻辑部分；设计时序电路也需要先设计好组合逻辑的激励函数。
- 第 5 章寄存器与计数器：移位寄存器的组合逻辑控制电路；计数器的进位链和使能逻辑均为本章知识的直接应用。
- 第 6 章半导体存储器：ROM（只读存储器）本质上是一个大型译码器加上存储阵列，其地址译码器就是本章介绍的二进制译码器；RAM 的片选和读写控制逻辑也是组合电路。
- 第 7 章可编程逻辑器件：PLA（可编程逻辑阵列）和 PAL（可编程阵列逻辑）直接基于本章的与或式逻辑结构；FPGA 的 LUT 实现原理已在 3.7 节介绍，理解 LUT 是学习 FPGA 设计的基础。

总之，组合逻辑电路是贯穿整个数字系统课程的核心基础，每一个后续章节都是在本章基础上的扩展与深化。务必深入理解并熟练掌握本章的核心内容。

3.10 工程实践注记

3.10.1 数字电路调试的基本方法

完成组合逻辑电路的设计和搭建之后，调试（Debug）是必不可少的环节。在实验室中调试组合逻辑电路的基本流程如下：首先进行静态功能测试——逐一输入各种测试向量（通常对小规模电路可以穷举所有输入组合），用示波器或逻辑分析仪（Logic Analyzer）观察各节点的电平状态，对照真值表验证输出是否正确。发现错误时，从输出端向输入端逐级追溯，找到第一个与理论值不符的节点，即为故障所在位置。

对于中等规模电路，穷举测试不现实，需要基于故障模型选取有代表性的测试向量。常用的策略是使用「一位固定故障」模型下的测试向量集，通常覆盖率可达 90%以上。在 FPGA 开发中，可以使用内置的逻辑分析器（如 Xilinx 的 ILA，Intel 的 Signal Tap）对内部信号进行在线实时捕获，极大地提高了调试效率——无需外部探头即可观察数百个内部信号。

静态调试之后是动态测试（时序测试），验证电路在高速工作时是否满足时序要求。动态测试需要配合时钟和时序激励源，用示波器观察关键信号的波形，检查是否存在建立/保持时间违例、毛刺等时序问题。在 FPGA 设计中，静态时序分析（STA）报告能提前预测时序问题，在流片（Tape-out）到 ASIC 时，时序签核（Timing Sign-off）是流片前最关键的步骤之一。

3.10.2 容差设计与裕量分析

实际工程中，器件参数存在制造偏差（Process Variation）、温度变化（Temperature Variation）和电源电压波动（Voltage Variation），简称 PVT 变化。一个在标准条件（nominal conditions）下工作正常的电路，在最恶劣 PVT 条件下（低温/高温、低压/高压、慢工艺角/快工艺角）可能出现功能失效或时序违例。

因此，在进行时序分析时，工程师必须在最坏情况（Worst Case）条件下验证建立时间约束，在最好情况（Best Case）条件下验证保持时间约束，这称为「工艺角分析（Corner Analysis）」。对于时序裕量（Slack），工程规范通常要求所有路径在最坏条件下仍保有不小于 10%时钟周期（或绝对值 0.5ns 以上）的正裕量，以保证芯片在整个生命周期内的可靠工作。

在 FPGA 设计中，EDA 工具（如 Xilinx Vivado）默认使用最慢工艺角（Slow Corner）进行时序分析，并自动报告最坏情况下的裕量，工程师只需确保所有路径的 Slack 均为正值即可。在 ASIC 设计中，需要在多个工艺角（如 SS-0°C-0.9V、FF-125°C-1.1V 等）分别跑 STA，满足全部约束才能签核。

3.10.3 低功耗组合逻辑设计技术

随着芯片集成度和工作频率的持续提升，功耗成为现代数字系统设计的核心约束之一。对于 CMOS 组合逻辑电路，动态功耗（Dynamic Power）是主要来源，其表达式为：

$P = \alpha \cdot C \cdot VDD^2 \cdot f$，其中 α 为活动因子（节点翻转概率），C 为节点电容，VDD 为电源电压，f 为工作频率。

常用的低功耗组合逻辑优化策略：

- 降低电源电压（Voltage Scaling）：由于功耗与 VDD^2成正比，将电源电压从 1.2V 降低到 0.9V 可将动态功耗降低约 44%，是最有效的降功耗手段。但降压会增加传播延迟，需要在功耗和速度之间权衡。
- 时钟门控（Clock Gating）：当某个功能模块不工作时，关闭其时钟信号，使其内部触发器和驱动的组合逻辑不再翻转，可减少动态功耗 30%~60%。时钟门控是现代 SoC（片上系统）设计中最广泛使用的低功耗技术。
- 操作数隔离（Operand Isolation）：在数据路径中，当某个运算单元的计算结果不被使用时，通过插入门控逻辑（如与门）将操作数置为固定值（如全零），避免运算单元因无效数据翻转而消耗动态功耗。
- 逻辑深度优化（Logic Depth Optimization）：减少组合逻辑的级数，可以在同样延迟下降低工作频率（即提高裕量），或者在相同频率下减少流水线级数，从而降低功耗。

在手机 SoC（如高通骁龙、苹果 A 系列芯片）中，这些低功耗技术被大量综合运用，使得芯片在提供极高性能的同时，功耗控制在几瓦特以内，保证了合理的电池续航。理解这些技术，是从学校走向工业界、成为高级数字设计工程师的重要一步。

3.10.4 组合逻辑电路的工程规范

在工业界的数字设计项目中，除了功能正确和时序满足之外，还需要遵守一系列工程规范（Engineering Guidelines），以确保设计的可维护性、可复用性和团队协作效率。常见的组合逻辑设计规范包括：

- 命名规范：信号名应直观反映其功能，低有效信号名末尾加'_n'或'_b'（如 cs_n 表示低有效片选）；时钟信号以 clk 开头，复位信号以 rst 开头；避免使用拼音缩写。
- 注释规范：每个模块（module）开头需有功能说明注释，包括输入输出端口说明和设计意图；复杂逻辑处需要行内注释解释设计原因；参数和常数应有单位和取值范围说明。
- 代码风格规范：组合逻辑用 assign 语句或 always @(*) 块描述；时序逻辑用 always @(posedge clk) 块描述；两者严格分开，不在同一个 always 块中混用；case 语句必须包含 default 分支。
- 仿真规范：每个设计模块必须有配套的测试平台（Testbench），覆盖正常功能测试和边界条件测试；仿真必须在功能正确后再进行综合，避免「仿真通过就流片」的错误做法。

养成良好的工程规范习惯，对于长期从事数字设计工作至关重要。一个清晰规范的设计，在几个月或几年后需要修改维护时，能大幅降低理解成本和出错概率；在团队协作中，规范的代码能提高代码复审（Code Review）效率，减少集成问题。

1. 题 3-1（多数判决分析）：逐级推导：P_1=(AB)'，P_2=(AC)'，P_3=(BC)'，Y=（$P_1P_2P_3$）'。展开：Y=AB+AC+BC。功能：三变量多数判决，2 个或 3 个输入为 1 时输出 1。

2. 题 3-5（四变量多数判决设计）：四个变量中至少 3 个为 1。最小项集合：Σm(7,11,13,14,15)。卡诺图化简：Y = ABC + ABD + ACD + BCD。与非门实现：Y = ((ABC)'·(ABD)'·(ACD)'·(BCD)')'，需 4 个三输入与非门加 1 个四输入与非门。

3. 题 3-7（用 74HC138 实现逻辑函数）：F(A,B,C)=Σm(0,2,3,6,7)。接线：A_2=A，A_1=B，A_0=C，S_1=1，$\bar{S}_2$=$\bar{S}_3$=0。将输出$\bar{Y}_0$、$\bar{Y}_2$、$\bar{Y}_3$、$\bar{Y}_6$、$\bar{Y}_7$接到一个五输入与非门，输出即为 F。

4. 题 3-8（用 74HC151 实现逻辑函数）：F(A,B,C)=Σm(1,2,4,7)。接线：S_2=A，S_1=B，S_0=C，$\bar{E}$N=0。D_1=1，D_2=1，D_4=1，D_7=1，其余 Di=0。输出 Y 即为 F。降维法（A 接 S_2，B 接 S_1，C 作为数端）：对每个 AB 组合分析 F 关于 C 的关系，D_0=C（AB=00 时 F=C），D_1=C'（AB=01 时 F=C'），D_2=1（AB=10 时 F=1），D_3=C（AB=11 时 F=C）。

5. 题 3-10（竞争冒险分析）：Y=AB'+A'B（异或）。当 A=0,B=0→1 切换时，A'B 和 AB' 均有路径，存在 1 型冒险。消除方法：由于异或不存在增加冗余项的简单方案，应在输出端加滤波电容，或在同步系统中用时钟锁存，确保采样时刻在稳定期内。

3.10.6 扩展阅读建议

本章内容是数字逻辑设计的经典基础，如需深入学习可参阅以下方向：

- 器件手册：德州仪器（TI）网站提供 74HC 系列全系列数据手册（免费下载），包含详细的电气参数、时序图、应用电路，是工程实践的第一手资料。
- EDA 工具：Xilinx Vivado 和 Intel Quartus Prime 均提供免费学生版，可用于学习 FPGA 设计流程；ModelSim（Mentor Graphics，现为 Siemens）是业界标准 HDL 仿真工具，同样提供学生版。
- 在线资源：MIT OCW 的 6.004「Computation Structures」课程（免费）提供了从逻辑门到处理器设计的完整视频和习题；FPGA4Fun 网站提供大量 FPGA 设计实例，适合动手实践。

数字逻辑电路的学习必须理论与实践并重。强烈建议在学习本章理论知识的同时，在实验板（如 TI LaunchPad、Xilinx Basys3 FPGA 开发板）上亲自搭建并调试本章介绍的各类电路，将纸面上的逻辑符号转化为真实可测量的电信号，这是理解数字电路的最直接、最有效的方式。

3.6 典型设计综合实例

前几节分别介绍了组合逻辑电路的分析方法、设计方法以及各类常用 MSI 器件的工作原理。本节通过四个具有代表性的综合设计实例，将上述内容融会贯通，展示从功能需求到完整电路的全过程，培养系统性设计思维。

3.6.1 格雷码与二进制码的相互转换电路

格雷码（Gray Code）是一种每相邻两个码字之间只有一位不同的二进制编码。这一特性在旋转编码器等场合极有价值：当被测量连续变化时，相邻状态间只有一位跳变，从根本上消除了多位同时翻转时的竞争冒险和错误读数。4 位格雷码与二进制码的对应关系遵循如下规律：

1. 二进制码转格雷码（B→G）

最高位格雷码等于最高位二进制码，其余各位等于相邻两位二进制码的异或：

电路实现：最高位直连，其余三位各需一个二输入异或门，共 3 个异或门，各输出独立计算，只需一级门延迟。这是效率极高的编码方案，广泛用于旋转编码器输出处理和数模转换器的量化编码。

验证（以十进制 5 为例）：二进制 $B_3B_2B_1B_0 = 0101$，格雷码 $G_3 = 0$，$G_2 = 0 \oplus 1 = 1$，$G_1 = 1 \oplus 0 = 1$，$G_0 = 0 \oplus 1 = 1$，故格雷码 = 0111。查表验证：十进制 5 的格雷码确实是 0111，正确。

2. 格雷码转二进制码（G→B）

最高位二进制码等于最高位格雷码，其余各位须从高位向低位逐位递推：

展开后可以看出，每一位二进制码等于从最高位格雷码起到当前位格雷码的全部异或：

电路同样需要 3 个异或门，但这三个门必须串联（B_2的输出是 B_1的输入，B_1的输出是 B_0的输入），关键路径经过 3 级异或门，延迟约为 B→G 转换的 3 倍。在时钟频率超过 30MHz 的系统中，G→B 转换可能成为时序瓶颈，此时可考虑利用查找表（ROM 或 LUT）直接实现转换，将多级串联变为一级查表，从而大幅缩短延迟。

3.6.2 奇偶校验电路

奇偶校验是数字通信和存储系统中最简单的错误检测方案，通过在数据中附加一位校验位，使整体数据中 1 的个数满足约定的奇偶性，接收方通过检验这一奇偶性来判断是否发生了奇数个比特错误。

1. 偶校验位发生器

设 4 位数据为 $D_3D_2D_1D_0$，偶校验要求附加校验位 P 后，五位数据中 1 的总数为偶数。校验位的表达式为：

当数据中有偶数个 1 时 P=0（无需补 1），有奇数个 1 时 P=1（补 1 使总数变偶）。电路实现：用 3 个二输入异或门构成异或树（$D_3 \oplus D_2$、$D_1 \oplus D_0$，再对两个结果进行异或），仅需 3 个异或门，是一种极为简洁的组合逻辑电路。

2. 校验检测器

接收端对收到的全部 5 位（含校验位）进行异或：

若无错误，5 位数据中 1 的个数为偶数，E=0；若发生奇数个比特翻转，E=1，触发错误指示。检测器同样由 3 个异或门实现（现在有 5 个输入，需 4 个异或门）。整个奇偶校验方案共需 7 个二输入异或门（3 个发生器+4 个检测器），却能可靠检测所有单比特错误，在存储器 ECC（Error Correcting Code）设计中是最基础的构件之一。

奇偶校验的根本局限：只能检测奇数个比特同时翻转（单比特错误最常见），无法检测偶数个同时翻转，也无法定位错误位置。如需纠错能力，须采用汉明码等更复杂的编码方案。

3.6.3 用 74HC138 实现全部三变量逻辑函数

3 线-8 线译码器 74HC138 能在一片芯片内同时展开三变量的全部 8 个最小项。将不同的最小项组合接到不同的外部与非门，理论上可以同时实现多个三变量函数，而无需重复使用译码器资源。这在需要多个相关联的三变量输出的场合（如三变量真值表的多个输出列）具有极大的面积优势。

例如，假设需要同时实现以下三个三变量函数：

三个函数共用一片 74HC138（A_0=A，A_1=B，A_2=C），分别用一个 4 输入与非门、一个 4 输入与非门和一个 4 输入与非门即可同时实现，总共仅需一片 74HC138 加三个与非门，比三套独立实现的方案节省约 60%的器件。这种资源共享的设计思路，在从 MSI 到 FPGA 的各类数字设计层次中都具有重要价值。

3.6.4 数值比较器实现地址范围检测

在嵌入式系统和计算机组成原理实验中，经常需要判断 CPU 的地址总线值是否落在某个特定的地址范围内，从而产生对应存储芯片或外设的片选信号。这是数值比较器的典型工程应用场景。

设计要求：8 位地址总线 ADDR[7:0]，当 ADDR 在十六进制 0x40~0x7F（二进制 01000000~01111111）范围内时，产生片选信号 CS=1。

分析：ADDR 的高两位 ADDR[7:6] 必须等于 01（即 ADDR[7]=0 且 ADDR[6]=1），低 6 位 ADDR[5:0] 可以是任意值。因此：

仅需一个与门和一个非门即可实现，无需任何比较器。这个例子说明：在地址译码的实际工程中，精心选择地址分配方案（使片选条件尽量简单），往往比使用复杂的比较电路更高效。这种从系统层面优化硬件设计的思维方式，称为设计协同（Design Co-optimization）。

若地址范围不规则（如 0x30~0x4F），才需要使用两片 74HC85 级联的 8 位比较器进行范围判断，分别检测 ADDR ≥ 0x30 和 ADDR ≤ 0x4F，将两个比较结果相与得到片选信号。在 FPGA 设计中，这类比较通常直接用 Verilog 的关系运算符（>=、<=）描述，由综合工具自动生成最优的比较逻辑。

3.7 组合逻辑电路的时序分析基础

在理论分析中，我们把逻辑门视为无延迟的理想器件。然而实际电路中每个门都存在传播延迟，这是影响数字系统工作速度的根本制约因素。本节介绍组合逻辑时序分析的基本概念，为后续学习时序逻辑电路打下基础。

3.7.1 传播延迟与关键路径

传播延迟（Propagation Delay，tpd）定义为从输入信号发生 50%幅值跳变，到输出信号达到 50%幅值的时间间隔。分为 tpHL（输出从高变低）和 tpLH（输出从低变高）两个参数，取较大值作为最坏情况延迟。

对于多级组合逻辑电路，从任意输入端到任意输出端之间，所有信号传播路径中延迟最大的一条称为关键路径（Critical Path）。电路总传播延迟等于关键路径的延迟之和。在同步数字系统中，为保证触发器能在时钟沿正确采样，组合逻辑的关键路径延迟必须满足：

其中 Tclk 是时钟周期，tsetup 是触发器建立时间，tskew 是时钟偏斜。这个不等式称为建立时间约束（Setup Constraint），是数字系统时序设计的核心不等式。违反建立时间约束意味着电路无法在给定时钟频率下正确工作，必须通过缩短关键路径（逻辑优化）或降低时钟频率来解决。

3.7.2 各类组合逻辑的延迟特性

不同类型的组合逻辑电路，因内部逻辑级数不同，传播延迟差异显著：

电路类型	典型逻辑级数	延迟特点	优化手段
基本逻辑门（与、或、非）	1	最小（约 7~10ns）	无需优化
异或门（XOR）	相当于 3~4 级简单门	中等（约 8~12ns）	可用 LUT 直接实现

行波进位加法器（n 位）	约 2n 级	随位数线性增长，n=32 时约 64ns	分组 CLA，降至 O(log n)
超前进位加法器（4 位）	约 5~6 级	固定（约 22ns），不随位数增长	多片级联仍有效
3 线-8 线译码器	3~4 级	中等（约 17ns）	FPGA 中用 LUT 优化
8 选 1 MUX	4~5 级	中等（约 18ns）	用 ROM 或 LUT 替代
4 位比较器	4~5 级	中等（约 21ns）	串行比较可做成流水线

从表中可见，行波进位加法器随位数增加延迟线性增长，这是其最大缺陷；其他 MSI 器件的延迟基本固定，在设计中可以直接查表确定其对系统时序的贡献。

3.7.3 减小传播延迟的设计策略

当关键路径延迟超出时序约束时，可以采用以下策略进行优化：

20. 逻辑层级优化：减少关键路径上的逻辑级数。例如将三级与或非结构改为两级与非-与非结构（与非-与非变换），或者利用布尔代数等价变换，将深树型结构改为宽平衡二叉树型结构（如 Wallace 树乘法器）。
21. 超前进位技术：在加法器设计中，用超前进位逻辑替代行波进位，将进位延迟从 O(n)降低到 O(log n)。74HC283 就是 4 位超前进位加法器，8 位以上时需要对进位组本身也做超前处理（两级超前进位）。
22. 流水线设计：在组合逻辑的中间点插入寄存器，将长延迟的组合逻辑切割成多个短延迟的阶段，每个阶段在一个时钟周期内完成。流水线不能减小单次计算的延迟（反而增加了 1~2 个时钟周期的通道延迟），但能显著提高吞吐量（每个时钟周期完成一次计算）。
23. 用查找表（LUT）替代多级逻辑：在 FPGA 中，任意 4~6 变量的组合逻辑函数都可以用单个 LUT 实现（固定延迟约 0.2~0.5ns），远比多级门级联快。在 ASIC 中，类似地可用 ROM（ROM-based Logic）实现大规模的逻辑映射。

3.8 习题详解精选

本节对本章部分习题给出完整的解题思路和关键步骤，帮助读者掌握解题方法。

3.8.1 例题精解一：多输出卡诺图化简

题目：用卡诺图化简以下两个具有相同输入（A、B、C、D）的逻辑函数，要求共享公共乘积项：

分析：F_1 是所有 D=1 的最小项之和，化简后 $F_1 = D$；F_2 是所有 D=0 的最小项之和，化简后 $F_2 = \overline{D}$。两个函数互为反变量，且可共享 D 这条连线（$F_1 = D$，$F_2 = \overline{D}$），整个电路仅需一根连线和一个非门，无需任何与门或或门。这体现了多输出化简的精髓：通过整体观察，往往能发现比分别化简更简洁的公共结构。

3.8.2 例题精解二：超前进位原理计算

题目：对于 4 位超前进位加法器，设输入 A=1010（十进制 10），B=0111（十进制 7），Cin=1，计算每一位的生成信号 G_i、传播信号 P_i，以及所有进位 C_1~C_4，验证最终结果。

解题过程：

第 0 位（i=0）：$A_0=0$，$B_0=1$，Cin=1；$G_0=A_0 \cdot B_0=0 \cdot 1=0$；$P_0=A_0 \oplus B_0=0 \oplus 1=1$；$S_0=P_0 \oplus Cin=1 \oplus 1=0$；$C_1=G_0+P_0 \cdot Cin=0+1 \cdot 1=1$。

第 1 位（i=1）：$A_1=1$，$B_1=1$；$G_1=1 \cdot 1=1$；$P_1=1 \oplus 1=0$；$S_1=P_1 \oplus C_1=0 \oplus 1=1$；$C_2=G_1+P_1 \cdot C_1=1+0 \cdot 1=1$。

第 2 位（i=2）：$A_2=0$，$B_2=1$；$G_2=0 \cdot 1=0$；$P_2=0 \oplus 1=1$；$S_2=P_2 \oplus C_2=1 \oplus 1=0$；$C_3=G_2+P_2 \cdot C_2=0+1 \cdot 1=1$。

第 3 位（i=3）：$A_3=1$，$B_3=0$；$G_3=1 \cdot 0=0$；$P_3=1 \oplus 0=1$；$S_3=P_3 \oplus C_3=1 \oplus 1=0$；$C_4=G_3+P_3 \cdot C_3=0+1 \cdot 1=1$。

最终结果：进位输出 Cout=C_4=1，本位和 $S_3S_2S_1S_0$=0010，因此最终结果为 1·0010（二进制）=18（十进制）。验证：10+7+1=18，正确。注意所有进位 C_1~C_4 的计算可以并行进行，无需等待低位进位的串行传播，这正是超前进位的核心优势。

3.8.3 例题精解三：竞争冒险分析与消除

题目：对逻辑函数 Y=(A+B)(A'+C) 进行竞争冒险分析，并消除冒险。

第一步，将函数展开为最简与或式：

注意 AA'=0 已被消去。现在检查最简与或式 Y=AC+A'B+BC 中是否存在冒险：

- 检查 AC 和 A'B：一个含 A，另一个含 A'，令使这两个乘积项同时趋于 1 的条件：AC=1 需要 A=1 且 C=1；A'B=1 需要 A=0 且 B=1。这两个条件无法同时成立（A 不能同时为 0 和 1），因此这对乘积项不存在冒险。
- 检查 AC 和 BC：均含 C，但无互补关系，不产生冒险。
- 检查 A'B 和 BC：均含 B，但无互补关系，不产生冒险。

结论：Y=AC+A'B+BC 不存在竞争冒险。这是因为三个乘积项中没有任何一对包含某变量及其互补，故展开后的与或式是无冒险式。实际上，对于原始形式 (A+B)(A'+C)，当 B=C=1 时，Y=(A+1)(A'+1)=1·1=1，A 变化时 Y 恒为 1，不存在冒险。

若将原函数改为 Y=AC+A'B（去掉 BC 项），则在 B=C=1 时 Y=A+A'，A 切换时产生冒险，增加冗余项 BC 后恢复为 Y=AC+A'B+BC，消除冒险。这再次验证了：冗余项 BC 正是保证 B=C=1 时 Y 不因 A 的竞争而产生毛刺的关键。

$$y_i = F_i(x_1, x_2, ..., x_n), \quad i = 1,2, ..., m$$

$$P_1 = \overline{AB}, \quad P_2 = \overline{BC}, \quad P_3 = \overline{AC}$$

$$Y = \overline{P_1 \cdot P_2 \cdot P_3} = \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}}$$

$$Y = AB + BC + AC$$

$$P = A \oplus B \quad （\mathrm{XOR}_1输出）$$

$$S = P \oplus C_{in} = A \oplus B \oplus C_{in} \quad （\mathrm{XOR}_2输出）$$

$$G_1 = A \cdot B, \quad G_2 = P \cdot C_{in} = (A \oplus B) \cdot C_{in}$$

$$C_{out} = G_1 + G_2 = AB + (A \oplus B) \cdot C_{in} = AB + AC_{in} + BC_{in}$$

$$P_1 = \overline{A + A} = \overline{A} = A', \quad P_2 = \overline{B + B} = B'$$

$$Y = \overline{P_1 + P_2} = \overline{A' + B'}$$

$$Y = \overline{A'} \cdot \overline{B'} = A \cdot B$$

$$Y = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC = \sum m(3,5,6,7)$$

$$Y = AB + BC + AC$$

$$Y = AB + BC + AC = \overline{\overline{AB} \cdot \overline{BC} \cdot \overline{AC}}$$

$$Y = \sum m(10,11,12,13,14,15) = \sum m\left(A \quad B \quad CD: 1010 \sim 1111\right)$$

$$Y = AB + AC$$

$$Y = A(B + C)$$

$$Y = \sum m(3,5,6,7,9,10) + \sum d(11,12,13,14,15)$$

$$Y = AB + CD + AC + BD$$

$$Y = AB + CD = \overline{\overline{AB + CD}} = \overline{\overline{AB} \cdot \overline{CD}}$$

$$Y_2 = I_4 + I_5 + I_6 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_0 = I_1 + I_3 + I_5 + I_7$$

$$Y = \sum m(i_1, i_2, ..., i_k) = \overline{\bar{Y}_{i_1} \cdot \bar{Y}_{i_2} \cdots \bar{Y}_{i_k}}$$

$$Y = \overline{\bar{Y}_1 \cdot \bar{Y}_3 \cdot \bar{Y}_5 \cdot \bar{Y}_6} = m_1 + m_3 + m_5 + m_6$$

$$Y = \overline{EN} \cdot \sum_{i=0}^{2^n-1} m_i\,(S_{n-1}, ..., S_1, S_0) \cdot D_i$$

$$Y = \overline{EN} \cdot (D_0\bar{S}_2\bar{S}_1\bar{S}_0 + D_1\bar{S}_2\bar{S}_1S_0 + \cdots + D_7S_2S_1S_0)$$

$$Y = F(S_2, S_1, S_0) = \sum_{i=0}^{7} f\,(m_i) \cdot m_i$$

$$D_0 = 0,\ D_1 = 1,\ D_2 = 1,\ D_3 = 0,\ D_4 = 1,\ D_5 = 0,\ D_6 = 0,\ D_7 = 1$$

$$S = A \oplus B, \quad C = A \cdot B$$

$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + (A \oplus B) \cdot C_{in} = AB + AC_{in} + BC_{in}$$

$$t_{总} \approx n \cdot t_{FA}$$

$$G_i = A_i \cdot B_i \quad （第i位必然产生进位）$$

$$P_i = A_i \oplus B_i \quad （第i位可能传播进位）$$

$$C_{i+1} = G_i + P_i \cdot C_i = G_i + P_i\left(G_{i-1} + P_{i-1}(G_{i-2} + \cdots)\right)$$

$$K = C_{out} + S_3S_2 + S_3S_1 \quad （K=1 时需要加 6 修正）$$

$$(A > B) = A\bar{B}, \quad (A = B) = \overline{A \oplus B} = AB + \bar{A}\bar{B}, \quad (A < B) = \bar{A}B$$

$$Y = AB + A'C + BC$$

$$Y = AC + BC' = \sum m(3,5,6,7) \quad （三变量，A/B/C）$$

$$Y = AC + BC' + AB$$

$$t_{pd,\max} = \max_{\text{all paths}} \sum_{i \in \text{path}} t_{pd,i}$$

$$t_{pd,\max} < T - t_{setup} - t_{skew}$$

$$t_{RCA,32} = 32 \times t_{FA} = 32 \times 2\text{ns} = 64\text{ns}$$

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$t_{CLA,4} \approx 2 \times t_{gate} \approx 4\text{ns} \quad （比 4 位 RCA 快约一倍）$$

$$G_3 = B_3, \quad G_2 = B_3 \oplus B_2, \quad G_1 = B_2 \oplus B_1, \quad G_0 = B_1 \oplus B_0$$

$$B_3 = G_3, \quad B_2 = B_3 \oplus G_2, \quad B_1 = B_2 \oplus G_1, \quad B_0 = B_1 \oplus G_0$$

$$B_2 = G_3 \oplus G_2, \quad B_1 = G_3 \oplus G_2 \oplus G_1, \quad B_0 = G_3 \oplus G_2 \oplus G_1 \oplus G_0$$

$$A - B = A + \overline{B} + 1$$

$$\text{SUB} = 0\text{:}\ Y = A + B, \quad \text{SUB} = 1\text{:}\ Y = A + \overline{B} + 1 = A - B$$

$$\text{OVF} = C_n \oplus C_{n-1}$$

$$\text{EQ} = E_3 \cdot E_2 \cdot E_1 \cdot E_0$$

$$\text{GT} = A_3\overline{B_3} + E_3A_2\overline{B_2} + E_3E_2A_1\overline{B_1} + E_3E_2E_1A_0\overline{B_0}$$

$$\text{LT} = \overline{A_3}B_3 + E_3\overline{A_2}B_2 + E_3E_2\overline{A_1}B_1 + E_3E_2E_1\overline{A_0}B_0$$

$$\text{EQ} = \overline{(A_3 \oplus B_3) + (A_2 \oplus B_2) + (A_1 \oplus B_1) + (A_0 \oplus B_0)}$$

$$G_3 = B_3, \quad G_2 = B_3 \oplus B_2, \quad G_1 = B_2 \oplus B_1, \quad G_0 = B_1 \oplus B_0$$

$$B_3 = G_3, \quad B_2 = B_3 \oplus G_2, \quad B_1 = B_2 \oplus G_1, \quad B_0 = B_1 \oplus G_0$$

$$B_k = G_3 \oplus G_2 \oplus \cdots \oplus G_k = \oplus_{i=k}^{3} G_i$$

$$P = D_3 \oplus D_2 \oplus D_1 \oplus D_0$$

$$E = P \oplus D_3 \oplus D_2 \oplus D_1 \oplus D_0$$

$$F_1 = \sum m(1,3,5,7) = C \quad （奇数最小项之和即等于 C）$$

$$F_2 = \sum m(3,5,6,7) = AB + BC + AC \quad \text{（多数判决）}$$

$$F_3 = \sum m(0,1,6,7) = \overline{A \oplus B} \quad \text{（A 同或 B）}$$

$$\mathrm{CS} = \overline{ADDR_7} \cdot ADDR_6$$

$$t_{pd,\max} \le T_{clk} - t_{setup} - t_{skew}$$

$$F_1 = \sum m(1,3,5,7,9,11,13,15)$$

$$F_2 = \sum m(0,2,4,6,8,10,12,14)$$

$$Y = (A+B)(A'+C) = AA' + AC + A'B + BC = AC + A'B + BC$$